Predicting Daily Demand for Capital Bikeshare

Regression Model Comparison and Temporal Robustness Testing

MLN601 Assessment 3 - CRISP-DM

Design and Creative Technologies, Torrens University

- **Student:** Luis Guilherme de Barros Andrade Faria - A00187785
- **Subject:** Machine Learning (MLN601)
- **Lecturer:** Dr. Kamran Shaukat
- **Assessment:** 3
- **Date:** August 2026

Field	Scope
Dataset	UCI Bike Sharing, Capital Bikeshare daily and hourly files, Washington DC, 2011-2012
Target	`cnt` : system-wide daily rentals
Primary question	Conditional daily-demand estimation from calendar attributes and expected weather
Secondary check	Rolling day-ahead temporal robustness, trained in 2011 and evaluated in 2012
Method	Regression comparison within CRISP-DM (Chapman et al., 2000)

1. Business Understanding

1.1 Business objective

Capital Bikeshare is a docked bicycle-sharing system (Capital Bikeshare, n.d.). This project asks:

> **Given calendar attributes and expected weather conditions, how accurately can regression models estimate Capital Bikeshare's system-wide daily rental demand?**

Daily demand is total network rentals in one day. An accurate estimate could support fleet-capacity, staffing and maintenance planning. Station inventory and rebalancing are outside scope because the CSVs lack station identifiers, locations and capacity; station-level data could extend the work.

Lime in Sydney is a contemporary dockless analogy that motivates my interest and makes the assessment closer to my reality. Although its operations differ, it raises a parallel question: how could calendar and weather patterns inform fleet planning in a local system? It motivates the work but is not model evidence.

1.2 Data and prediction scope

Each row represents one system-wide day, with `cnt` as total rentals. Calendar attributes are known in advance. Observed weather stands in for the forecast a real user would hold, so deployment would require forecast inputs. The hourly file supports exploration without changing the daily target.

The primary random 75/25 experiment estimates demand for days resembling 2011-2012 using calendar and weather without past demand. The secondary experiment tests transfer through time: selection uses 2011, then counts through day D plus next-day conditions predict D+1 in 2012. Forecasting all of 2012 on 1 January is separate because later counts would be unavailable.

This report therefore runs three comparisons. The first two answer the assessment question; the third tests whether that answer transfers forward in time.

Table 1 - What this report compares

#	Comparison	Split	Compared against	Question answered	Where
1	Model selection	Random 75/25, training partition only	The other 11 family and feature-set configurations	Which configuration predicts most accurately?	Table 6
2	Value of modelling	Random 75/25, holdout opened once	A constant training-mean prediction	Does the selected model beat not modelling at all?	Tables 7 and 8
3	Forward transfer	Time-ordered: trained in 2011, scored on 2012	Naive lag-1, lag-7 and rolling seven-day rules	Does the approach still work on a period it has not seen?	Table 12

Comparisons 1 and 2 address the brief. Comparison 3 is an additional check that limits how the result may be used.

1.3 Evaluation criteria

MAE is primary because it reports the average miss in rentals per day. RMSE gives more weight to large errors, while R-squared shows explained variation relative to predicting the sample mean.

The training-mean baseline tests whether predictors add value; the primary model must improve MAE by 40%. Rolling seven-day demand is the temporal reference because it adapts to recent demand; the frozen ML model must beat it by 5% on identical 2012 dates.

Model, feature and hyperparameter choices use training-only cross-validation. Lowest MAE leads; configurations within 5% are separated by RMSE and then simplicity. The holdout is used only after selection.

These study-defined criteria came from earlier exploratory versions and are not stakeholder-validated service levels.

1.4 Assumptions and limitations

- Observed weather omits forecast error, making reported day-ahead accuracy optimistic.
- Two years provide one year-on-year transition, weak evidence for persistent growth.
- Weather category 4 is absent; an input guard must restrict use to supported categories 1-3.
- System totals cannot answer station questions. The Institute for Transportation and Development Policy (ITDP) publishes bikeshare planning guidance; the National Association of City Transportation Officials (NACTO) publishes network and station-siting guidance. They explain why future station planning also needs identifiers, capacity, trip flows, transit connections and local demand (Institute for Transportation & Development Policy [ITDP], 2018; National Association of City Transportation Officials [NACTO], 2016).

2. Data Understanding

2.1 Source, loading and granularity

The UCI Bike Sharing dataset supplies `day.csv` and `hour.csv` for Capital Bikeshare in 2011-2012 (Fanaee-T & Gama, 2014; University of California, Irvine, n.d.). The daily file is the modelling source. The hourly file is loaded separately for intraday EDA and cross-file validation.

In [1]:

```
import os
import warnings
from pathlib import Path

os.environ["PYTHONWARNINGS"] = "ignore"
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.base import clone
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    train_test_split, GridSearchCV, KFold, TimeSeriesSplit, cross_validate
)
from sklearn.inspection import permutation_importance
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

sns.set_theme(style="whitegrid")
RANDOM_STATE = 42

NB_DIR = Path.cwd()
BASE_DIR = NB_DIR.parent if NB_DIR.name == "notebook" else NB_DIR
DATA_DIR = BASE_DIR / "dataset"
OUTPUT_DIR = BASE_DIR / "outputs"
FIG_DIR = OUTPUT_DIR / "figures"
FIG_DIR.mkdir(parents=True, exist_ok=True)

day_df = pd.read_csv(DATA_DIR / "day.csv", parse_dates=
["dteday"]).sort_values("dteday").reset_index(drop=True)
hour_df = pd.read_csv(DATA_DIR / "hour.csv", parse_dates=
["dteday"]).sort_values(["dteday", "hr"]).reset_index(drop=True)
print("day.csv :", day_df.shape, day_df.dteday.min().date(), "to",
day_df.dteday.max().date())
print("hour.csv:", hour_df.shape, hour_df.dteday.min().date(), "to",
hour_df.dteday.max().date())
```

```
day.csv : (731, 16) 2011-01-01 to 2012-12-31
hour.csv: (17379, 17) 2011-01-01 to 2012-12-31
```

In [2]:
```
hourly_daily = hour_df.groupby("dteday", as_index=False)
["cnt"].sum().rename(columns={"cnt": "hourly_sum"})
cross_file = day_df[["dteday", "cnt"]].merge(hourly_daily, on="dteday",
how="outer", validate="one_to_one")
cross_file["difference"] = cross_file["hourly_sum"] - cross_file["cnt"]
assert len(cross_file) == 731
assert cross_file["difference"].eq(0).all(), "Hourly cnt does not sum to daily
cnt for every date"
hours_per_date = hour_df.groupby("dteday").size()
incomplete_dates = int(hours_per_date.lt(24).sum())
omitted_hour_rows = int((24 - hours_per_date).sum())
assert incomplete_dates == 76 and omitted_hour_rows == 165

granularity = pd.DataFrame([
    {"file": "day.csv", "rows": len(day_df), "observation": "one daily
prediction unit",
     "role": "selected for modelling"},
    {"file": "hour.csv", "rows": len(hour_df), "observation": "correlated
hourly observations",
     "role": "intraday understanding and cross-file validation"},
])
print("Table 2 - Granularity and role")
display(granularity)
print(f"Cross-file validation passed: hourly cnt sums exactly to daily cnt on
all {len(cross_file)} dates.")
print(f"Hourly panel audit: {incomplete_dates} dates have fewer than 24 rows; "
      f"{omitted_hour_rows} zero-demand hour rows are omitted in total.")
```

Table 2 - Granularity and role

	file	rows	observation	role
0	day.csv	731	one daily prediction unit	selected for modelling
1	hour.csv	17379	correlated hourly observations	intraday understanding and cross-file validation

```
Cross-file validation passed: hourly cnt sums exactly to daily cnt on all 731 dat
es.
Hourly panel audit: 76 dates have fewer than 24 rows; 165 zero-demand hour rows a
re omitted in total.
```

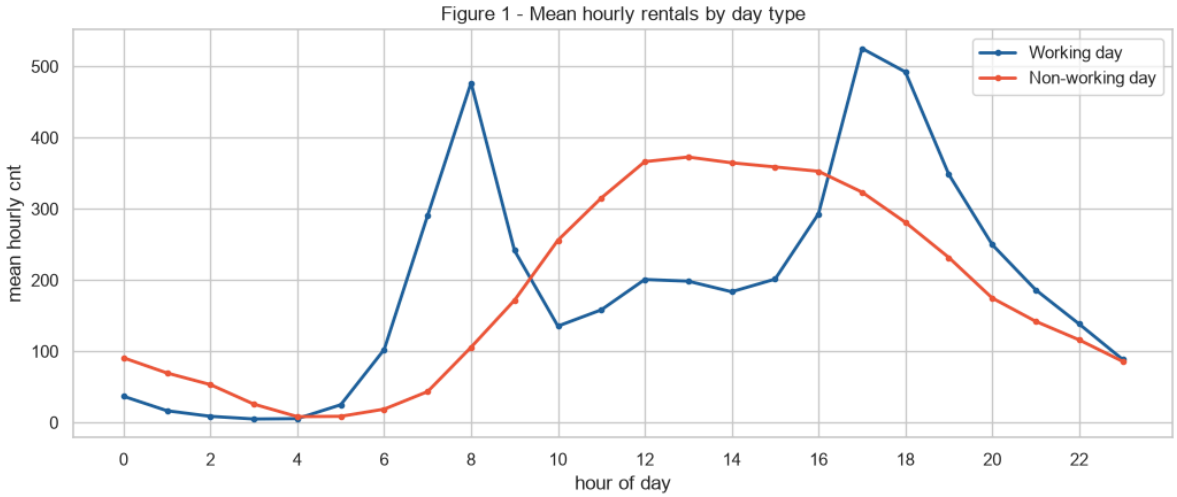
`hour.csv` is unbalanced: 76 of 731 dates have fewer than 24 observations, with 165 zero-demand rows omitted. An hourly model would change granularity, weight dates unequally and omit those periods. Selecting `day.csv` gives every target day one complete observation **to avoid hourly bias**. The hourly counts still reconstruct all daily totals and remain useful for intraday interpretation.

In [3]:
```
hour_profile = (hour_df.groupby(["workingday", "hr"], as_index=False)
["cnt"].mean())
peaks = (hour_profile.loc[hour_profile.groupby("workingday")["cnt"].idxmax()]
         .assign(day_type=lambda x: x["workingday"].map({0: "Non-working day",
1: "Working day"}))
         [["day_type", "hr", "cnt"]].rename(columns={"hr": "peak_hour", "cnt":
"mean_cnt"}))
peaks["mean_cnt"] = peaks["mean_cnt"].round(1)
print("Table 3 - Peak hourly demand")
display(peaks)
assert int(peaks.loc[peaks.day_type == "Working day", "peak_hour"].iloc[0]) ==
17
assert np.isclose(peaks.loc[peaks.day_type == "Working day",
"mean_cnt"].iloc[0], 525.3)
assert int(peaks.loc[peaks.day_type == "Non-working day", "peak_hour"].iloc[0])
== 13
assert np.isclose(peaks.loc[peaks.day_type == "Non-working day",
"mean_cnt"].iloc[0], 372.7)

fig, ax = plt.subplots(figsize=(10.5, 4.6))
for flag, label, color in [(1, "Working day", "#20639B"), (0, "Non-working
day", "#ED553B")]:
    p = hour_profile[hour_profile.workingday == flag]
    ax.plot(p.hr, p.cnt, marker="o", ms=3, lw=2, label=label, color=color)
ax.set(title="Figure 1 - Mean hourly rentals by day type", xlabel="hour of
day", ylabel="mean hourly cnt")
ax.set_xticks(range(0, 24, 2)); ax.legend()
plt.tight_layout(); plt.savefig(FIG_DIR / "v5_hourly_demand.png", dpi=160);
plt.show()
```

Table 3 - Peak hourly demand

	day_type	peak_hour	mean_cnt
13	Non-working day	13	372.7
41	Working day	17	525.3



Working-day demand peaks at 17:00 (525.3 rentals), while non-working-day demand peaks at 13:00 (372.7). The profiles are consistent with commute-oriented and broader daytime use, respectively; aggregate counts do not prove trip purpose.

In [4]:
```
span_days = (day_df.dteday.max() - day_df.dteday.min()).days + 1
norm_cols = ["temp", "atemp", "hum", "windspeed"]
quality = pd.DataFrame([
    ("Daily rows equal calendar span", len(day_df) == span_days, f"
{len(day_df)} rows / {span_days} days"),
    ("No date gaps", day_df.dteday.diff().dropna().dt.days.max() == 1, "daily
continuity"),
    ("No exact duplicates", day_df.duplicated().sum() == 0, f"
{day_df.duplicated().sum()} duplicates"),
    ("No source missing values", day_df.isna().sum().sum() == 0, f"
{day_df.isna().sum().sum()} missing"),
    ("casual + registered = cnt", (day_df.casual + day_df.registered ==
day_df.cnt).all(), "all rows"),
    ("Normalised weather in [0,1]", all(day_df[c].between(0, 1).all() for c in
norm_cols), "all four columns"),
    ("Humidity above zero", (day_df.hum > 0).all(), f"{int((day_df.hum ==
0).sum())} zero reading"),
    ("Daily weather categories supported", set(day_df.weathersit.unique()) ==
{1, 2, 3}, "category 4 absent"),
], columns=["check", "pass", "evidence"])
print("Table 4 - Data quality audit")
display(quality)

zero_hum_date = day_df.loc[day_df.hum.eq(0), "dteday"].iloc[0]
zero_hour_rows = hour_df[hour_df.dteday.eq(zero_hum_date)]
print("Zero-humidity date:", zero_hum_date.date(), "| hourly rows:",
len(zero_hour_rows),
      "| hourly humidity values:", sorted(zero_hour_rows.hum.unique()))

# Keep a raw frame for sensitivity analysis, then mark the implausible value
missing.
raw_day_df = day_df.copy()
day_df.loc[day_df.hum.eq(0), "hum"] = np.nan
assert day_df.hum.isna().sum() == 1
```

```
Table 4 - Data quality audit
```

	check	pass	evidence
0	Daily rows equal calendar span	True	731 rows / 731 days
1	No date gaps	True	daily continuity
2	No exact duplicates	True	0 duplicates
3	No source missing values	True	0 missing
4	casual + registered = cnt	True	all rows
5	Normalised weather in [0,1]	True	all four columns
6	Humidity above zero	False	1 zero reading
7	Daily weather categories supported	True	category 4 absent

```
Zero-humidity date: 2011-03-10 | hourly rows: 22 | hourly humidity values: [np.fl
oat64(0.0)]
```

The daily series is complete and `casual + registered = cnt` holds throughout. One date reports zero humidity across 22 hourly rows. It is treated as a recording fault, changed to missing and median-imputed inside each training fold; a sensitivity check tests the decision. Plausible tail values are retained.

Weather category 4 is absent, so the supported domain is 1-3 and still requires an input guard.

2.2 Variables and exploratory analysis

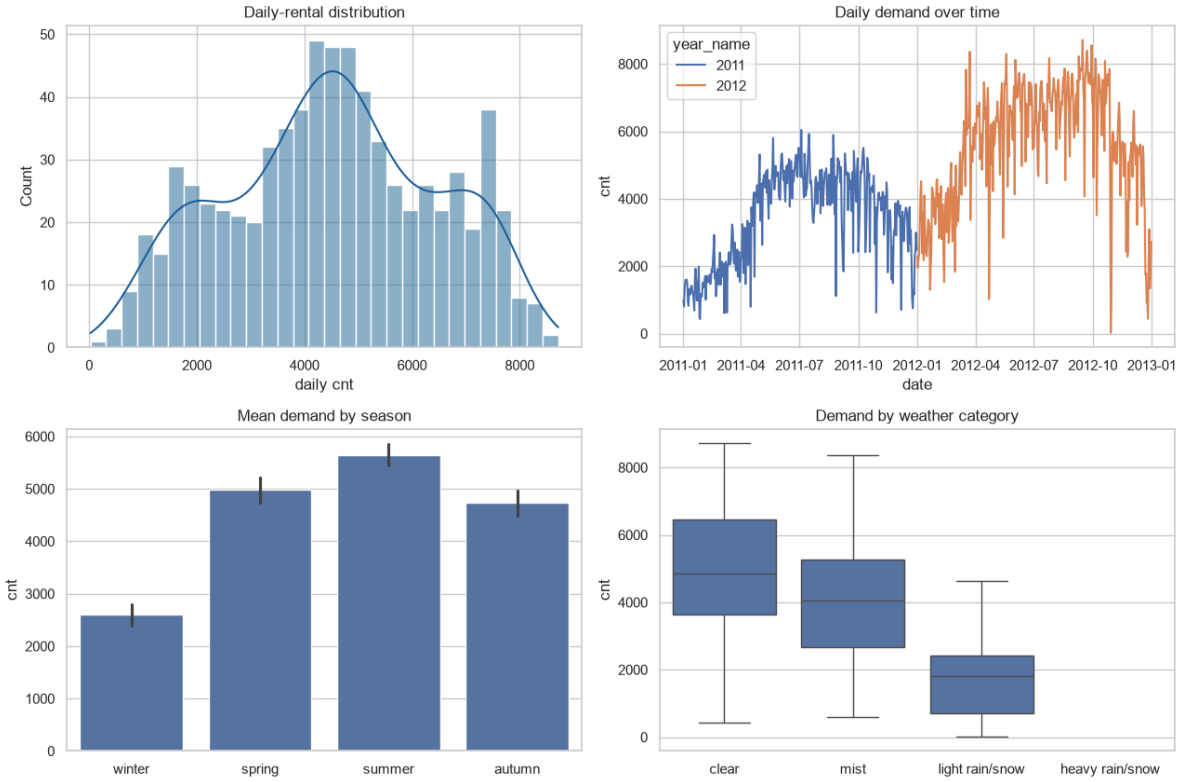
Variables	Role
`cnt`	Continuous target: system-wide rentals per day
`season`, `yr`, `mnth`, `holiday`, `weekday`, `workingday`	Calendar predictors
`weathersit`, `temp`, `atemp`, `hum`, `windspeed`	Weather predictors
`dteday`	Ordering and elapsed-time construction; not passed directly to a model
`casual`, `registered`	Descriptive plots only; excluded because they sum exactly to `cnt`

`casual` and `registered` describe user groups, not independent predictors. Registered users may include commuters, but membership status does not prove trip purpose.

In [5]:
```
season_lbl = {1: "winter", 2: "spring", 3: "summer", 4: "autumn"}
weather_lbl = {1: "clear", 2: "mist", 3: "light rain/snow", 4: "heavy
rain/snow"}
day_df["season_name"] = day_df.season.map(season_lbl)
day_df["weather_name"] = day_df.weathersit.map(weather_lbl)
day_df["year_name"] = day_df.yr.map({0: "2011", 1: "2012"})

fig, ax = plt.subplots(2, 2, figsize=(13, 9))
sns.histplot(day_df.cnt, bins=30, kde=True, ax=ax[0, 0], color="#20639B")
ax[0, 0].set(title="Daily-rental distribution", xlabel="daily cnt")
sns.lineplot(data=day_df, x="dteday", y="cnt", hue="year_name", ax=ax[0, 1])
ax[0, 1].set(title="Daily demand over time", xlabel="date", ylabel="cnt")
sns.barplot(data=day_df, x="season_name", y="cnt", order=["winter", "spring",
"summer", "autumn"], ax=ax[1, 0])
ax[1, 0].set(title="Mean demand by season", xlabel="")
sns.boxplot(data=day_df, x="weather_name", y="cnt", order=["clear", "mist",
"light rain/snow", "heavy rain/snow"], ax=ax[1, 1])
ax[1, 1].set(title="Demand by weather category", xlabel="")
fig.suptitle("Figure 2 - Daily demand, season and weather", y=1.01)
plt.tight_layout(); plt.savefig(FIG_DIR / "v5_daily_eda.png", dpi=150);
plt.show()
```

Figure 2 - Daily demand, season and weather



Demand rises across the two years, peaks in warmer seasons and falls in adverse weather.

A documentation correction. The dataset description states `season (1:springer, 2:summer, 3:fall, 4:winter)`. The dates disagree. Code 1 covers December to March at a mean of 12.2 degrees Celsius and the lowest mean demand of 2,604; code 3 covers June to September at 29.0 degrees and the highest mean demand of 5,644. The codes are therefore offset by one quarter from the published labels, and this notebook labels them `1 = winter, 2 = spring, 3 = summer, 4 = autumn` on the evidence of the calendar and temperature rather than the documentation. Nothing in the modelling changes, because `season` enters every pipeline as a categorical code and never as a name; the correction affects the readability of Figures 2 and 3 and of Table 13, and the wording of this section.

In [6]:

```
comp = day_df.groupby("season_name")[["casual",
"registered"]].mean().reindex(["winter", "spring", "summer", "autumn"])
fig, ax = plt.subplots(figsize=(8.5, 4.4))
comp.plot(kind="bar", stacked=True, ax=ax, color=["#ED553B", "#20639B"])
ax.set(title="Figure 3 - Mean casual and registered rentals by season",
       xlabel="", ylabel="mean daily rentals")
ax.tick_params(axis="x", rotation=0)
ax.legend(title="user group")
plt.tight_layout(); plt.savefig(FIG_DIR / "v5_composition.png", dpi=160);
plt.show()

# Each weather variable is plotted in its original unit, not the shared
normalised scale.
# day.csv divides temp by 41 C, atemp by 50 C, hum by 100 % and windspeed by 67
km/h, so a
# single normalised axis would give one x position four different physical
meanings.
WEATHER_UNITS = [
    ("temp", 41, "temperature (degrees Celsius)", "#20639B"),
    ("atemp", 50, "feels-like temperature (degrees Celsius)", "#3CAEA3"),
    ("hum", 100, "relative humidity (%)", "#F6D55C"),
    ("windspeed", 67, "wind speed (km/h)", "#ED553B"),
]
fig, ax = plt.subplots(1, 4, figsize=(16, 4.2), sharey=True)
for axis, (column, scale, label, color) in zip(ax, WEATHER_UNITS):
    sns.regplot(x=day_df[column] * scale, y=day_df.cnt, ax=axis, order=2,
ci=None,
                scatter_kws={"s": 10, "alpha": .3, "color": color},
                line_kws={"color": "black", "lw": 1.4})
    axis.set(xlabel=label, ylabel="")
ax[0].set(ylabel="daily cnt")
fig.suptitle("Figure 4 - Weather measures and daily demand, each in its
original unit "
             "(quadratic guide line, not a fitted model)", y=1.04)
plt.tight_layout(); plt.savefig(FIG_DIR / "v5_weather_units.png", dpi=160,
bbox_inches="tight"); plt.show()
```

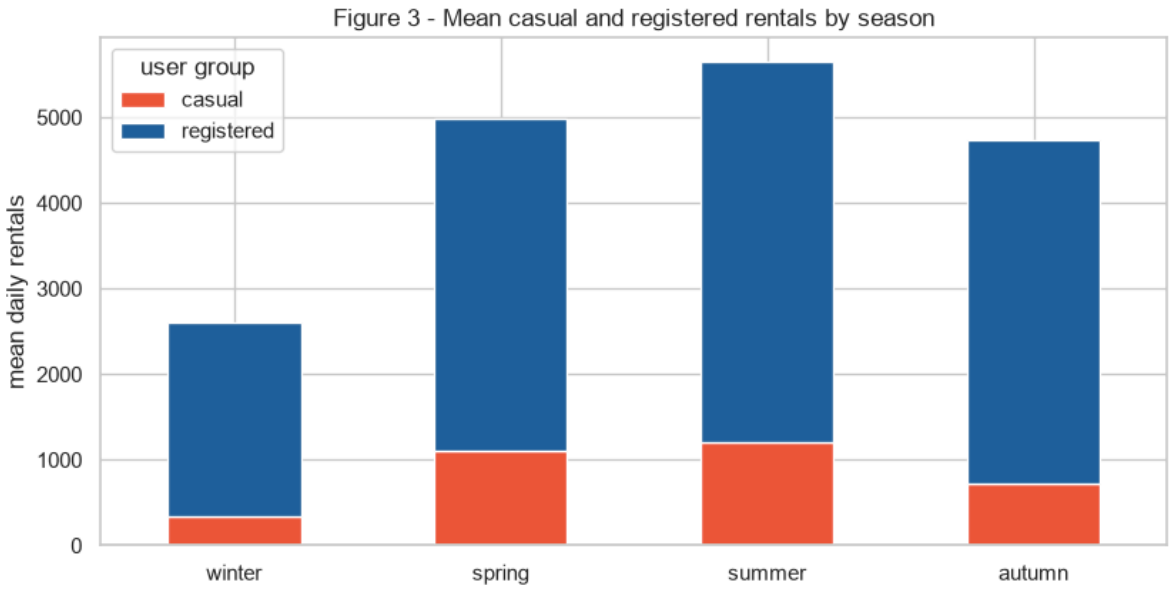
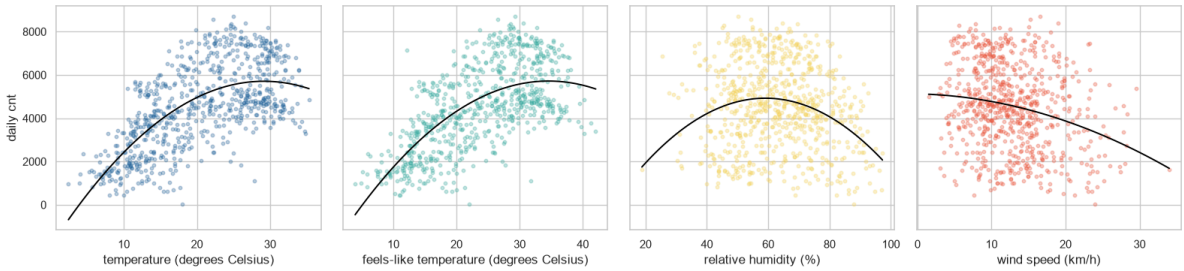


Figure 4 - Weather measures and daily demand, each in its original unit (quadratic guide line, not a fitted model)



Registered users provide most rentals in every season, while the casual share expands in the warmer ones. Both columns stay descriptive because they sum exactly to the target.

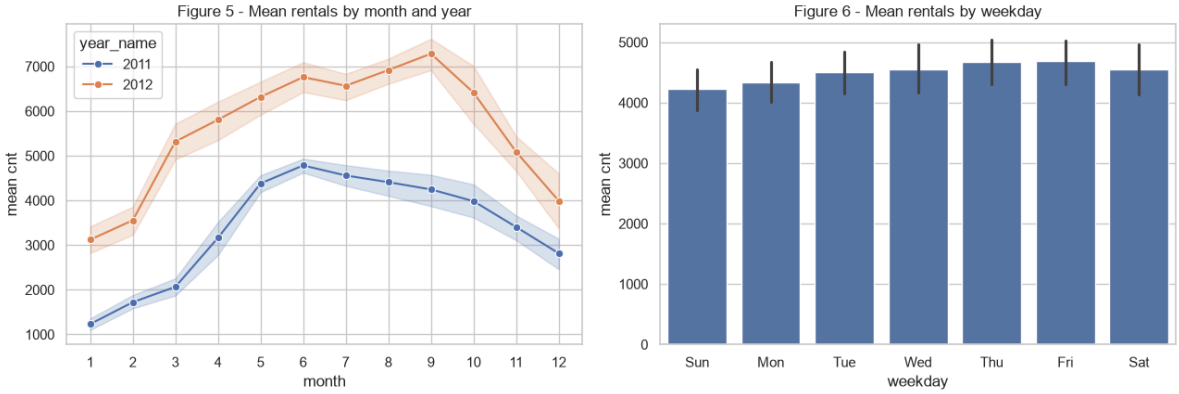
Plotting each weather variable on its own axis in real units matters for reading the strength of each relationship. Demand climbs steeply with temperature to roughly 30 degrees Celsius and then stops climbing, which is the curvature the pairplot also shows and the reason a squared temperature term is built in Section 3.3. Feels-like temperature repeats the same shape, as its correlation of +0.631 against +0.627 would suggest.

The other two behave differently from each other, which the shared axis in the earlier version of this figure concealed. Wind speed declines steadily across its range. Humidity does not: the guide line peaks near 60% and falls away at both ends, so the weak -0.115 correlation in Table 5 is summarising a non-monotonic pattern rather than a gentle slope, and the extremes are thinly populated. Neither variable separates high-demand from low-demand days on its own; the scatter at any given humidity or wind speed spans most of the observed range of `cnt`.

A single normalised axis would have hidden all of this by placing 20.5 degrees Celsius, 25 degrees Celsius, 50% humidity and 33.5 km/h at the same x position.

In [7]:
```
weekday_lbl = {0: "Sun", 1: "Mon", 2: "Tue", 3: "Wed", 4: "Thu", 5: "Fri", 6:
"Sat"}
day_df["weekday_name"] = day_df.weekday.map(weekday_lbl)

fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
sns.lineplot(data=day_df, x="mnth", y="cnt", hue="year_name", marker="o",
ax=ax[0])
ax[0].set(title="Figure 5 - Mean rentals by month and year", xlabel="month",
ylabel="mean cnt")
ax[0].set_xticks(range(1, 13))
sns.barplot(data=day_df, x="weekday_name", y="cnt",
            order=["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"], ax=ax[1])
ax[1].set(title="Figure 6 - Mean rentals by weekday", xlabel="weekday",
ylabel="mean cnt")
plt.tight_layout(); plt.savefig(FIG_DIR / "v5_month_weekday.png", dpi=160);
plt.show()
```



Monthly demand follows the seasonal wave and every 2012 month exceeds its 2011 counterpart. Weekday means vary much less, showing that aggregation blends working-day and leisure patterns visible in the hourly profiles.

In [8]:
```
corr_cols = ["season", "yr", "mnth", "holiday", "weekday", "workingday",
             "weathersit", "temp", "atemp", "hum", "windspeed", "cnt"]
corr_matrix = day_df[corr_cols].corr()
cnt_correlations = corr_matrix["cnt"].drop("cnt").sort_values(ascending=False)
assert cnt_correlations.index[0] == "atemp"
assert cnt_correlations.loc["weathersit"] < 0 and
cnt_correlations.loc["windspeed"] < 0

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", center=0,
            square=True, cbar_kws={"shrink": .8}, annot_kws={"size": 7})
plt.title("Figure 7 - Correlation matrix for daily variables")
plt.tight_layout(); plt.savefig(FIG_DIR / "v5_correlation_heatmap.png",
dpi=160); plt.show()
print("Table 5 - Correlations with cnt")
display(cnt_correlations.round(3).to_frame("correlation_with_cnt"))
```

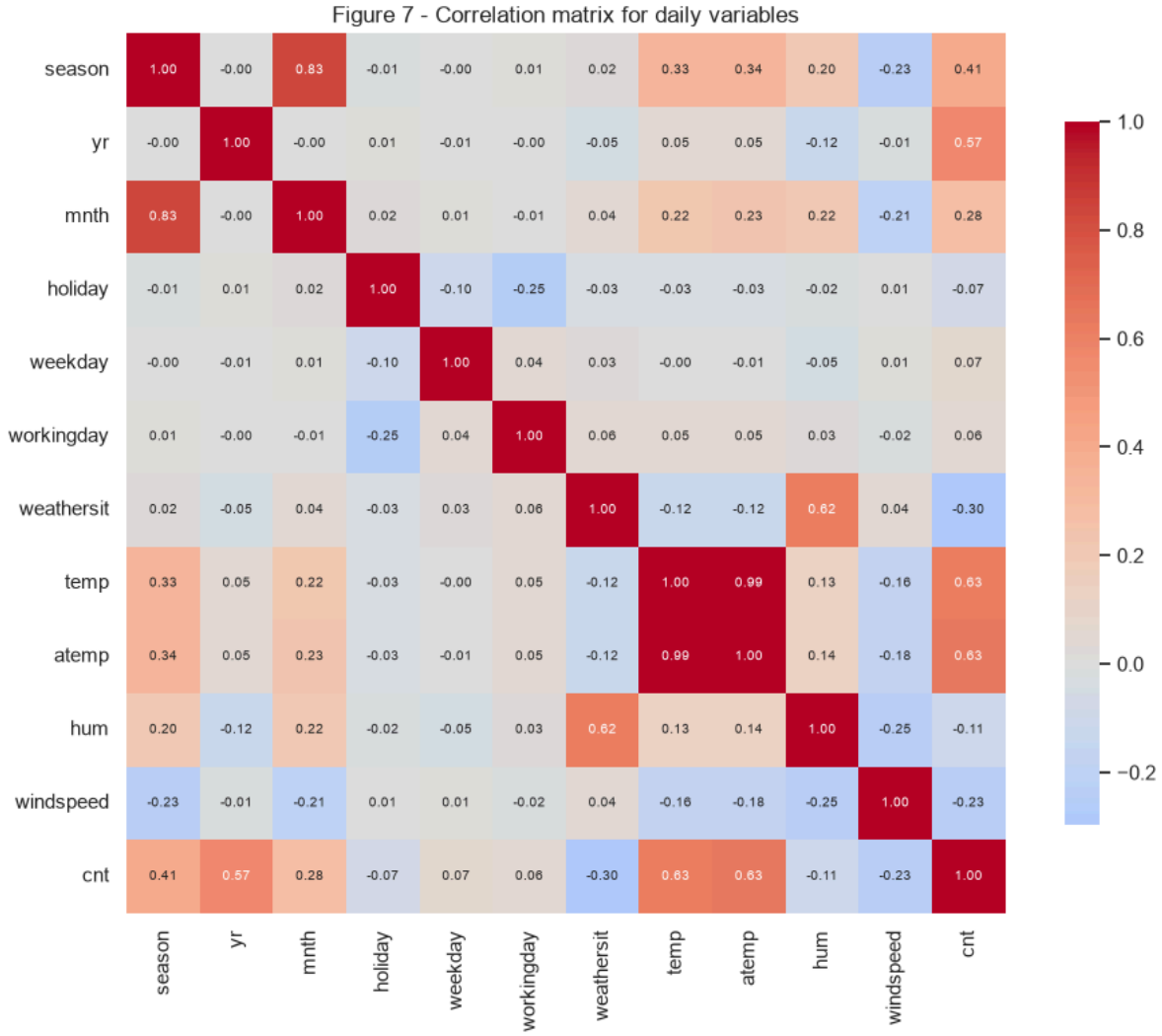


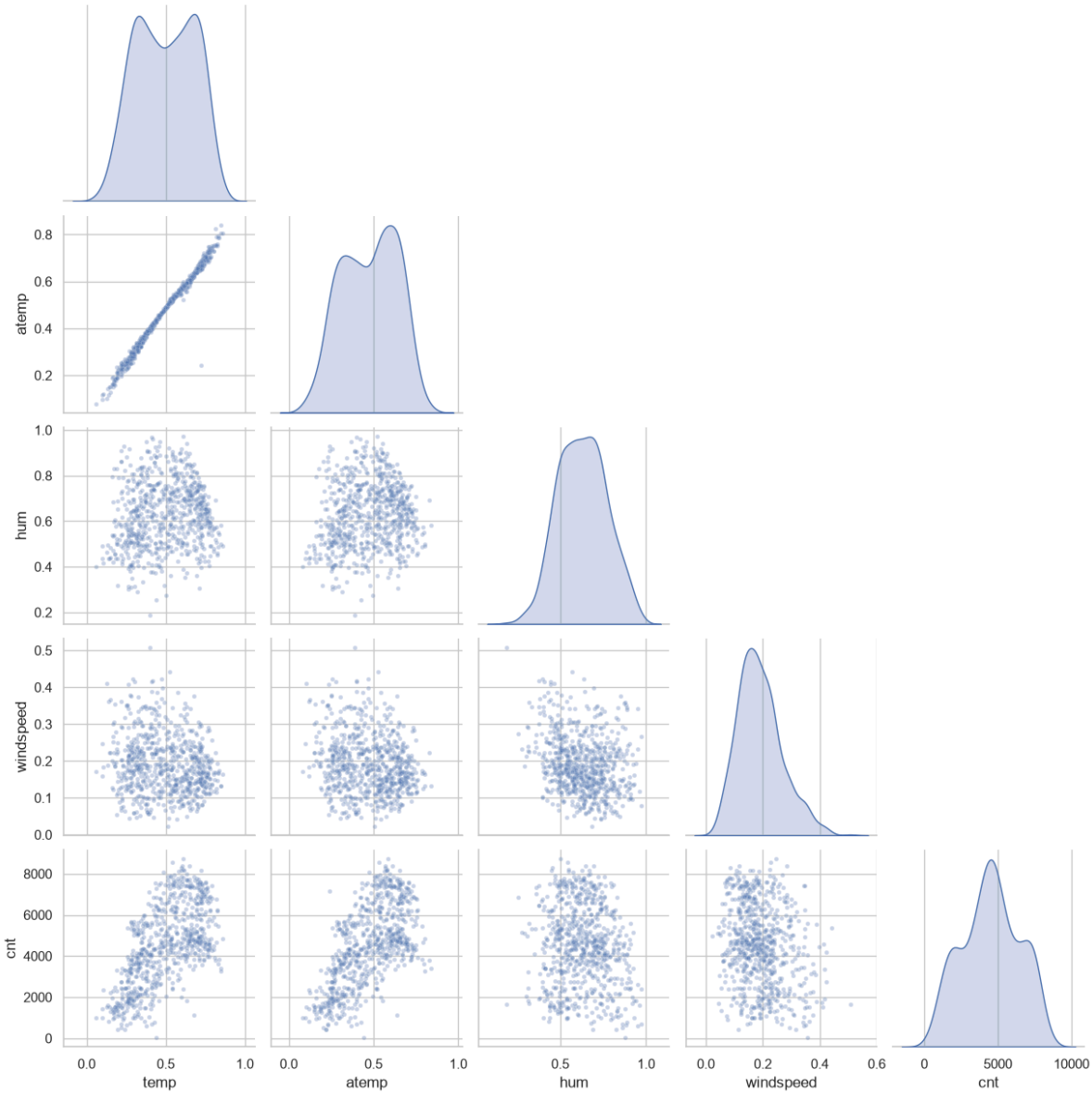
Table 5 - Correlations with cnt

	correlation_with_cnt
atemp	0.631
temp	0.627
yr	0.567
season	0.406
mnth	0.280
weekday	0.067
workingday	0.061
holiday	-0.068
hum	-0.115
windspeed	-0.235
weathersit	-0.297

`atemp` has the highest correlation with `cnt` (+0.631), followed by `temp` (+0.627) and `yr` (+0.567). Adverse weather (`weathersit`, -0.297), windspeed (-0.235) and corrected humidity (-0.115) are negative. The near-collinearity of `temp` and `atemp` cautions against interpreting linear coefficients independently; categorical codes also limit causal interpretation.

In [9]:
```
# Stage 2.6: use seaborn's pairplot function and include its output.
pairplot_data = day_df[["temp", "atemp", "hum", "windspeed", "cnt"]].dropna()
pair_grid = sns.pairplot(pairplot_data, corner=True, diag_kind="kde",
                         plot_kws={"alpha": .3, "s": 12})
pair_grid.fig.suptitle("Figure 8 - Seaborn pairplot of weather variables and
daily demand", y=1.02)
pair_grid.savefig(FIG_DIR / "v5_pairplot.png", dpi=140, bbox_inches="tight")
plt.show()
```

Figure 8 - Seaborn pairplot of weather variables and daily demand



The pairplot confirms positive, nonlinear relationships between demand and both temperature measures. Humidity and windspeed show weaker negative patterns and broad scatter. `temp` and `atemp` move almost identically, consistent with their high correlation, while no single weather variable fully explains demand.

3. Data Preparation

Section 2 reported what the data contains. This section records the decisions taken because of it, and the reasoning behind each one.

3.1 Selecting the data

Daily rather than hourly records. Both files describe the same system, so the choice of granularity is a modelling decision and not a formatting one. Section 2.1 showed that `hour.csv` is an unbalanced panel: 76 of the 731 dates carry fewer than 24 rows, and the 165 absent rows are zero-demand hours that were never written. Fitting on hourly records would therefore weight dates unequally, giving a complete summer date 24 observations and an incomplete winter date as few as 22, and it would never show the model the quiet hours that are missing. Selecting `day.csv` gives every target day exactly one complete observation and **avoids that hourly bias**. The hourly file is retained for intraday interpretation and for the cross-file check that reconstructs all 731 daily totals exactly.

Columns excluded on principle. `casual` and `registered` are dropped from every predictor list because they sum to `cnt` by construction; including either would let the model read part of its own answer. `dteday` is not passed to any estimator either, and is used only for ordering and for building elapsed time. `instant` is a row counter carrying no information about demand.

Two frames, so the temporal question does not shrink the primary sample. `primary_df` keeps all 731 days and contains no past-demand inputs. `temporal_df` begins after a causal seven-day warm-up and contains 724 days, because a seven-day rolling mean is undefined for the first week of 2011. Building one shared frame would have cost seven days from the primary experiment for no benefit to it.

3.2 Cleaning, and estimating the missing value

The daily file arrives with no missing cells, no duplicate rows and no gaps in the date sequence, so cleaning reduces to a single judgement call.

One humidity reading is not a low value, it is a fault. Section 2.1 recorded `hum = 0` on one date. Relative humidity of exactly zero does not occur in Washington DC, and cross-checking `hour.csv` confirms the diagnosis: all 22 hourly rows for that date also read zero, which is the signature of a failed sensor rather than of dry weather.

The value is estimated, not deleted. Deleting the row would have discarded a complete demand observation to fix one input, so the reading is marked missing and then estimated by the median of the training data. The estimation happens inside `SimpleImputer(strategy="median")` within every pipeline, so the median is refitted on each training fold and never sees the validation fold or the holdout. A single `fillna` computed over the whole frame would have been simpler and would have leaked a statistic of the test data into a training input.

One consequence is worth stating plainly. The interaction `atemp_hum` is built before imputation, so it carries the same missing value and the imputer fills it as a column in its own right. Its estimated value is the median interaction, not the product of the imputed humidity, which leaves the two very slightly inconsistent on that single row out of 731. Rebuilding the interaction after imputation, inside the pipeline, would remove the inconsistency; it was not done because Table 9 already bounds the entire humidity treatment at 3 rentals of holdout MAE, which is smaller than the effect being corrected.

The decision is checked rather than asserted. Table 9 refits the frozen primary model on the uncorrected data and reports both results. If the correction had been load-bearing, that table would say so.

No outlier removal. Extreme but physically plausible values are retained. Low-demand days are real events, mostly adverse weather, and removing them would flatter the error statistics while making the model worse at exactly the days a planner cares about.

3.3 Engineering, encoding and the split

What the correlations imply for preparation. `atemp` correlates with `cnt` at +0.631 and `temp` at +0.627, and the two move almost identically with each other. Both are retained, because the tree families are untroubled by collinear inputs and dropping one would discard information the ensembles use. The consequence is recorded instead: individual linear coefficients on `temp` and `atemp` are not interpretable in isolation, which is one reason Section 5.2 explains the model through permutation importance and TreeSHAP rather than through coefficients. `weathersit` (-0.297), `windspeed` (-0.235) and corrected `hum` (-0.115) are the negative correlates, and all three are kept as predictors.

Interaction and curvature terms. `atemp * hum` and `temp` squared are constructed so that the additive families can express what the scatter plots show: warm and humid days behave differently from warm and dry ones, and demand stops rising once temperature passes roughly 30 degrees Celsius. Both are built only from target-day exogenous inputs, so neither smuggles in past demand.

Cyclical encoding, in the temporal experiment only. Month, weekday and season are also encoded as sine and cosine pairs for the time-ordered sets. The reason is specific to that split: `OneHotEncoder(drop="first", handle_unknown="ignore")` encodes an unseen category as all zeros, which is byte-identical to the dropped reference category. Under a 2011-to-2012 split, a month the model never met would silently be treated as January. A sine and cosine pair has no unseen values and keeps December adjacent to January.

`yr` is excluded from the temporal predictors. It is constant at 0 across the whole 2011 training year, so the model can learn nothing from it, and it then arrives as a constant 1 in 2012, a value never observed during fitting. It stays in the primary feature sets, where both values appear on both sides of the split.

Autoregressive inputs are built causally. `lag_1_cnt`, `lag_7_cnt` and `roll_7_cnt` use only demand observed strictly before the target day; the rolling mean shifts first and then rolls, never the reverse. The code cell below asserts this for every row rather than trusting the expression, because shift-after-roll is the easiest way to leak a target into its own predictor.

The split, declared here and executed in Section 4. The primary experiment uses a single random 75/25 partition created once at `random_state=42`, giving 548 training and 183 holdout days. Selection of family, feature set and hyperparameters happens entirely inside the 548 training rows by cross-validation; the 183 holdout rows are scored once, after everything is frozen. The temporal experiment splits by calendar instead, fitting on 2011 and scoring on 2012.

In [10]:
```
# Shared calendar/weather engineering. Interactions use only target-day
exogenous inputs.
day_df["atemp_hum"] = day_df["atemp"] * day_df["hum"]
day_df["temp_sq"] = day_df["temp"] ** 2
day_df["days_since_start"] = (day_df.dteday - day_df.dteday.min()).dt.days
day_df["mnth_sin"], day_df["mnth_cos"] = np.sin(2*np.pi*day_df.mnth/12),
np.cos(2*np.pi*day_df.mnth/12)
day_df["weekday_sin"], day_df["weekday_cos"] =
np.sin(2*np.pi*day_df.weekday/7), np.cos(2*np.pi*day_df.weekday/7)
day_df["season_sin"], day_df["season_cos"] = np.sin(2*np.pi*day_df.season/4),
np.cos(2*np.pi*day_df.season/4)

primary_df = day_df.copy().reset_index(drop=True)

# Causal temporal features: every window ends before the target day.
day_df["lag_1_cnt"] = day_df.cnt.shift(1)
day_df["lag_7_cnt"] = day_df.cnt.shift(7)
day_df["roll_7_cnt"] = day_df.cnt.shift(1).rolling(7).mean()
assert day_df.lag_1_cnt.equals(day_df.cnt.shift(1))
assert day_df.lag_7_cnt.equals(day_df.cnt.shift(7))
for i in range(7, len(day_df)):
    assert np.isclose(day_df.loc[i, "roll_7_cnt"], day_df.loc[i-7:i-1,
"cnt"].mean())
temporal_df = day_df.dropna(subset=["roll_7_cnt"]).reset_index(drop=True)

assert len(primary_df) == 731
assert len(temporal_df) == 724
print("primary_df rows:", len(primary_df), "| temporal_df rows:",
len(temporal_df))
```

```
primary_df rows: 731 | temporal_df rows: 724
```

In [11]:

```
PRIMARY_NOMINAL = ["season", "mnth", "weekday", "weathersit"]
PRIMARY_BINARY = ["yr", "holiday", "workingday"]
WEATHER = ["temp", "atemp", "hum", "windspeed"]
CYCLICAL = ["mnth_sin", "mnth_cos", "weekday_sin", "weekday_cos", "season_sin",
"season_cos"]
INTERACTIONS = ["atemp_hum", "temp_sq"]
TREND = ["days_since_start"]
AUTOREGRESSIVE = ["lag_1_cnt", "lag_7_cnt", "roll_7_cnt"]

PRIMARY_FEATURE_SETS = {
    "core": PRIMARY_NOMINAL + PRIMARY_BINARY + WEATHER,
    "core + interactions": PRIMARY_NOMINAL + PRIMARY_BINARY + WEATHER +
INTERACTIONS,
    "core + interactions + trend": PRIMARY_NOMINAL + PRIMARY_BINARY + WEATHER +
INTERACTIONS + TREND,
}
TEMPORAL_FEATURE_SETS = {
    "core": ["weathersit", "holiday", "workingday"] + WEATHER + CYCLICAL,
    "core + interactions": ["weathersit", "holiday", "workingday"] + WEATHER +
CYCLICAL + INTERACTIONS,
    "core + interactions + autoregressive": ["weathersit", "holiday",
"workingday"] + WEATHER + CYCLICAL + INTERACTIONS + AUTOREGRESSIVE,
    "core + interactions + autoregressive + trend": ["weathersit", "holiday",
"workingday"] + WEATHER + CYCLICAL + INTERACTIONS + AUTOREGRESSIVE + TREND,
}

for feature_list in list(PRIMARY_FEATURE_SETS.values()) +
list(TEMPORAL_FEATURE_SETS.values()):
    assert "casual" not in feature_list and "registered" not in feature_list
assert all(c not in cols for cols in PRIMARY_FEATURE_SETS.values() for c in
AUTOREGRESSIVE)
assert all("yr" not in cols for cols in TEMPORAL_FEATURE_SETS.values())

def make_preprocessor(columns, temporal=False):
    nominal = ["weathersit"] if temporal else [c for c in PRIMARY_NOMINAL if c
in columns]
    binary_pool = ["holiday", "workingday"] + ([] if temporal else ["yr"])
    binary = [c for c in binary_pool if c in columns]
    continuous = [c for c in columns if c not in nominal + binary]
    return ColumnTransformer([
        ("nominal", OneHotEncoder(drop="first", handle_unknown="ignore"),
nominal),
        ("continuous", Pipeline([("impute", SimpleImputer(strategy="median")),
                                 ("scale", StandardScaler())]), continuous),
        ("binary", "passthrough", binary),
    ])

def pipeline_for(model, columns, temporal=False):
    return Pipeline([("prep", make_preprocessor(columns, temporal=temporal)),
("model", model)])

def metrics(y_true, predictions):
    mse = mean_squared_error(y_true, predictions)
    return {"MAE": mean_absolute_error(y_true, predictions), "MSE": mse,
            "RMSE": float(np.sqrt(mse)), "R2": r2_score(y_true, predictions)}

GRIDS = {
    "Linear Regression": (LinearRegression(), {}),
    "K-Nearest Neighbors": (KNeighborsRegressor(), {
        "model__n_neighbors": [3, 5, 7, 9, 11, 15], "model__weights":
["uniform", "distance"]}),
    "Random Forest": (RandomForestRegressor(random_state=RANDOM_STATE,
n_jobs=1), {
        "model__n_estimators": [200, 400], "model__max_depth": [None, 8, 16],
        "model__min_samples_leaf": [1, 2, 4]}),
```

```
    "Gradient Boosting": (GradientBoostingRegressor(random_state=RANDOM_STATE),
{
        "model__n_estimators": [200, 400], "model__max_depth": [2, 3],
        "model__learning_rate": [0.05, 0.1]}),
}

MODEL_SIMPLICITY = {"Linear Regression": 1, "K-Nearest Neighbors": 2,
                    "Random Forest": 3, "Gradient Boosting": 4}
FEATURE_SIMPLICITY = {name: i for i, name in enumerate(
    ["core", "core + interactions", "core + interactions + trend",
     "core + interactions + autoregressive", "core + interactions +
autoregressive + trend"], 1)}

def compact_params(params):
    if not params:
        return "default"
    labels = {"model__learning_rate": "lr", "model__max_depth": "depth",
              "model__n_estimators": "estimators", "model__min_samples_leaf":
"leaf",
              "model__n_neighbors": "neighbors", "model__weights": "weights"}
    return "; ".join(f"{labels.get(key, key)}={value}" for key, value in
params.items())

# Display-only column labels. The saved CSVs keep their long self-describing
names; the
# printed tables use short ones so that every column survives the width of a
PDF page.
# Labels avoid spaces on purpose: a printed table wraps at spaces first, so
"mean residual"
# would break across two lines while "mean_residual" holds together in the same
width.
NARROW = {
    "feature_set": "features", "best_params": "params", "selected_model":
"model",
    "cv_mean_MAE": "CV_MAE", "cv_std_MAE": "CV_MAE_SD",
    "cv_mean_RMSE": "CV_RMSE", "cv_std_RMSE": "CV_RMSE_SD",
    "holdout_MAE": "MAE", "holdout_RMSE": "RMSE", "holdout_R2": "R2",
    "mean_baseline_MAE": "baseline_MAE", "baseline_improvement_pct":
"vs_baseline_pct",
    "MAE_pct_of_holdout_mean_demand": "MAE_pct_of_mean",
    "train_rows": "train", "validation_rows": "val",
    "train_window": "train_window", "validation_window": "val_window",
    "largest_2012_prediction": "max_pred_2012", "2011_training_target_max":
"train_max_2011",
    "2012_actual_maximum": "actual_max_2012",
    "mean_residual_actual_minus_prediction": "mean_residual",
    "median_absolute_error": "median_AE", "days_closer": "days_won",
    "win_rate_excluding_ties_pct": "win_rate_pct", "tied_dates": "ties",
    "MAE_increase_mean": "MAE_increase", "MAE_increase_std": "SD",
    "mean_absolute_SHAP": "mean_abs_SHAP",
    "humidity_pct": "hum_pct", "mean reference": "mean_ref",
    "rolling-7 reference": "roll7_ref", "correlation_with_cnt":
"correlation_with_cnt",
}
ABBREV = {"Linear Regression": "LR", "K-Nearest Neighbors": "KNN",
          "Random Forest": "RF", "Gradient Boosting": "GB"}


def narrow(frame):
    """Shorten column labels for display only. Saved artefacts keep the long
names."""
    return frame.rename(columns=NARROW)


def choose_configuration(rows):
```

```
    frame = pd.DataFrame(rows).copy()
    best_mae = frame["cv_mean_MAE"].min()
    shortlist = frame[frame["cv_mean_MAE"] <= 1.05 * best_mae].copy()
    shortlist["simplicity"] = shortlist["model"].map(MODEL_SIMPLICITY) +
shortlist["feature_set"].map(FEATURE_SIMPLICITY) / 10
    return shortlist.sort_values(["cv_mean_RMSE", "simplicity",
"cv_mean_MAE"]).iloc[0]
```

Preparation leaves 731 primary rows and 724 temporal rows, with no observation deleted and one input value estimated. The assertions above are part of the deliverable rather than scaffolding: they prove that the rolling window closes before the target day, that no predictor list contains `casual` or `registered`, that past-demand inputs never reach the primary experiment, and that `yr` never reaches the temporal one. Each is a failure this design could plausibly have suffered, so each is checked by the notebook instead of being claimed by the report.

Every transformation that learns anything from the data, meaning median imputation and standard scaling, is defined inside the pipeline and not applied to the frame beforehand. That placement is what allows Section 4 to refit the whole chain on each cross-validation fold without a training statistic escaping into evaluation data.

4. Modelling

Four regression families are compared with lightweight grids in scikit-learn (Pedregosa et al., 2011). Linear Regression tests a global additive relationship; K-Nearest Neighbors tests local similarity; Random Forest averages decorrelated trees; Gradient Boosting sequentially corrects residuals (Friedman, 2001). Scaling and fold-fitted median imputation live inside every pipeline.

Both experiments draw on the same daily records and differ in how those records are split and which inputs are permitted. The primary experiment uses all 731 days; the temporal experiment uses the 724 that remain once the seven-day causal warm-up is excluded. Each produces one learned estimate and one reference estimate, giving the four columns compared in Table 13.

In [12]:
```
from IPython.display import Image, display as show_image

# Figure 9 is authored in Mermaid and rendered to PNG so that it survives the notebook,
# HTML and PDF exports identically. The source is kept in the repository alongside it.
show_image(Image(filename=str(FIG_DIR / "v5_pipeline.png"), width=760))
print("Figure 9 — How the two experiments produce the four estimates compared in Table 13")
```

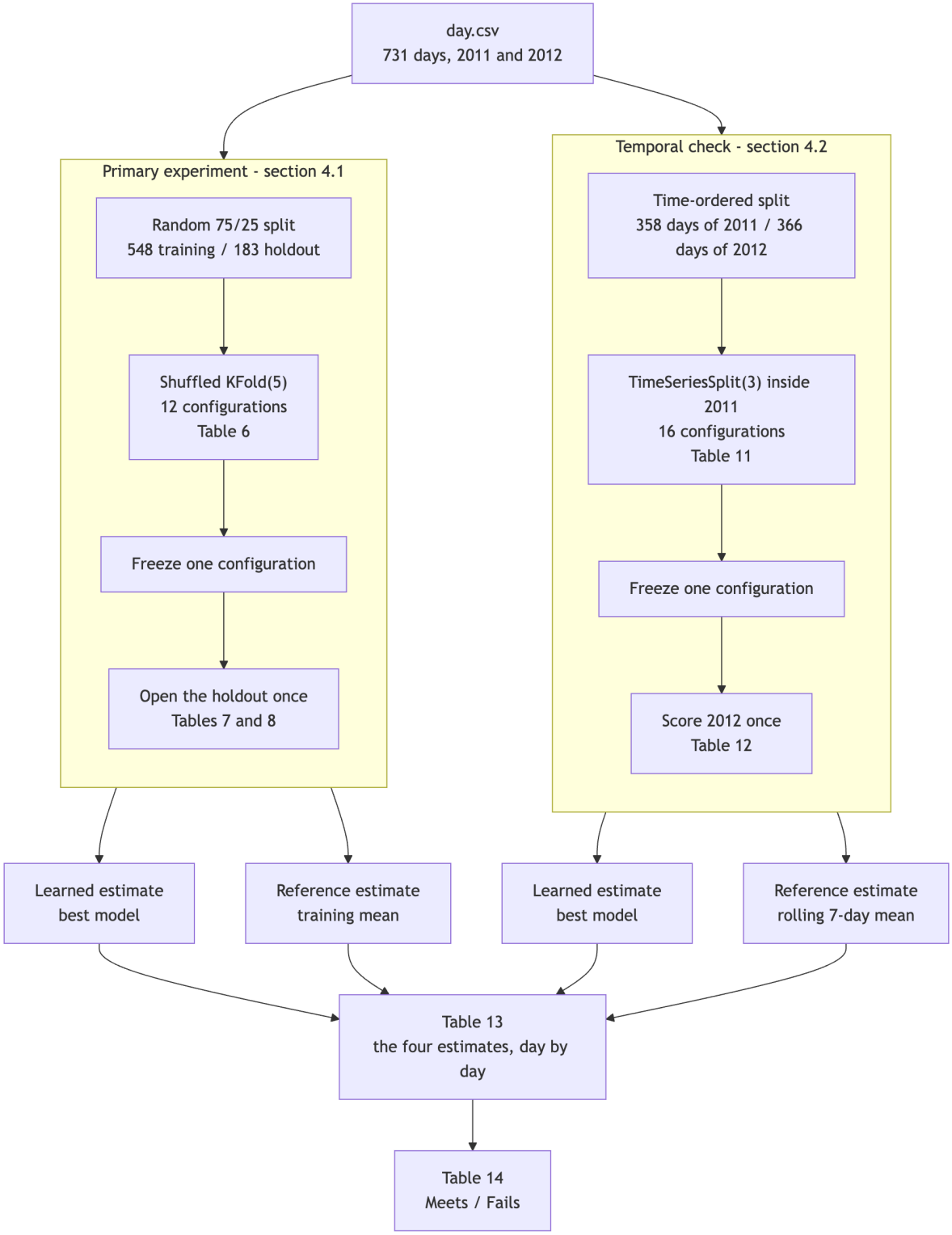


Figure 9 — How the two experiments produce the four estimates compared in Table 13

4.1 Primary model comparison: random 75/25 holdout

The split is created once with `random_state=42`. Only the 75% training partition participates in family, feature-set and hyperparameter selection. Each of the 12 family/feature-set combinations uses shuffled five-fold `KFold` and MAE scoring. The 25% holdout is accessed only after the global configuration and one configuration per family have been frozen.

In [13]:

```
all_indices = primary_df.index.to_numpy()
primary_train_idx, primary_holdout_idx = train_test_split(
    all_indices, test_size=0.25, random_state=RANDOM_STATE
)
assert set(primary_train_idx).isdisjoint(primary_holdout_idx)
assert len(primary_train_idx) == 548 and len(primary_holdout_idx) == 183

primary_train = primary_df.loc[primary_train_idx].copy()
primary_holdout = primary_df.loc[primary_holdout_idx].copy()
primary_cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
primary_rows, primary_estimators = [], {}

for feature_name, columns in PRIMARY_FEATURE_SETS.items():
    for model_name, (estimator, grid) in GRIDS.items():
        search = GridSearchCV(
            pipeline_for(clone(estimator), columns, temporal=False), grid,
            scoring="neg_mean_absolute_error", cv=primary_cv, n_jobs=1,
refit=True
        )
        search.fit(primary_train[columns], primary_train.cnt)
        rmse_cv = cross_validate(
            search.best_estimator_, primary_train[columns], primary_train.cnt,
            cv=primary_cv, scoring="neg_root_mean_squared_error", n_jobs=1
        )["test_score"]
        row = {
            "model": model_name, "feature_set": feature_name,
            "cv_mean_MAE": -search.best_score_,
            "cv_std_MAE": search.cv_results_["std_test_score"]
[search.best_index_],
            "cv_mean_RMSE": -rmse_cv.mean(), "cv_std_RMSE": rmse_cv.std(),
            "best_params": compact_params(search.best_params_),
        }
        primary_rows.append(row)
        primary_estimators[(model_name, feature_name)] = search.best_estimator_

primary_cv_table = (pd.DataFrame(primary_rows)
                    .sort_values(["cv_mean_MAE", "cv_mean_RMSE"])
                    .reset_index(drop=True))
primary_cv_table.to_csv(OUTPUT_DIR / "primary_cv_selection_v5.csv",
index=False)

primary_choice = choose_configuration(primary_rows)
primary_family_choices = {
    family: choose_configuration([r for r in primary_rows if r["model"] ==
family])
    for family in GRIDS
}
primary_winner_name = primary_choice["model"]
primary_winner_features = primary_choice["feature_set"]
primary_winner_columns = PRIMARY_FEATURE_SETS[primary_winner_features]
primary_winner = primary_estimators[(primary_winner_name,
primary_winner_features)]

# Leakage audit: choices contain CV statistics from training indices only; the
holdout has not
# been passed to fit, GridSearchCV, cross_validate or choose_configuration.
selection_indices = set(primary_train_idx)
assert selection_indices.isdisjoint(set(primary_holdout_idx))
assert len(primary_df) == 731
print("Table 6 — Primary CV selection (training partition only)")
display(narrow(primary_cv_table.round({"cv_mean_MAE": 1, "cv_std_MAE": 1,
                                       "cv_mean_RMSE": 1, "cv_std_RMSE": 1})))
print("Frozen primary configuration:", primary_winner_name, "|",
```

```
primary_winner_features,
      "|", primary_choice["best_params"])
```

Table 6 — Primary CV selection (training partition only)

	model	features	CV_MAE	CV_MAE_SD	CV_RMSE	CV_RMSE_SD	params
0	Gradient Boosting	core + interactions + trend	477.0	34.1	657.6	45.4	lr=0.1; depth=2; estimators=400
1	Random Forest	core + interactions + trend	498.0	18.1	723.5	38.5	depth=16; leaf=1; estimators=200
2	Gradient Boosting	core	513.8	56.4	690.8	53.2	lr=0.05; depth=2; estimators=400
3	Gradient Boosting	core + interactions	519.5	47.4	697.1	46.4	lr=0.05; depth=2; estimators=400
4	Random Forest	core	533.6	26.5	746.8	25.7	depth=16; leaf=1; estimators=400
5	Random Forest	core + interactions	539.1	24.3	752.6	23.2	depth=16; leaf=1; estimators=400
6	Linear Regression	core + interactions + trend	559.3	19.8	752.4	36.9	default
7	Linear Regression	core + interactions	560.4	19.3	752.6	38.5	default
8	K-Nearest Neighbors	core + interactions + trend	563.0	15.1	794.0	40.0	neighbors=7; weights=distance
9	Linear Regression	core	591.0	14.7	806.8	41.9	default
10	K-Nearest Neighbors	core + interactions	756.5	39.1	966.5	48.0	neighbors=5; weights=distance
11	K-Nearest Neighbors	core	760.7	42.7	957.3	52.9	neighbors=5; weights=distance

```
Frozen primary configuration: Gradient Boosting | core + interactions + trend | l
r=0.1; depth=2; estimators=400
```

In [14]:
```
# First use of the fixed 25% holdout: score one training-selected configuration
per family.
primary_holdout_rows = []
mean_model = DummyRegressor(strategy="mean").fit(
    np.zeros((len(primary_train), 1)), primary_train.cnt
)
baseline_pred = mean_model.predict(np.zeros((len(primary_holdout), 1)))
primary_holdout_rows.append({"model": "Mean baseline", "feature_set": "constant
training mean",
                             "best_params": "default",
**metrics(primary_holdout.cnt, baseline_pred)})

primary_family_predictions = {}
for family, choice in primary_family_choices.items():
    feature_name = choice["feature_set"]
    columns = PRIMARY_FEATURE_SETS[feature_name]
    fitted = primary_estimators[(family, feature_name)]
    pred = fitted.predict(primary_holdout[columns])
    primary_family_predictions[family] = pred
    primary_holdout_rows.append({"model": family, "feature_set": feature_name,
                                 "best_params": choice["best_params"],
                                 **metrics(primary_holdout.cnt, pred)})

primary_holdout_table = (pd.DataFrame(primary_holdout_rows)
                         .sort_values("MAE").reset_index(drop=True))
primary_holdout_table.to_csv(OUTPUT_DIR / "primary_holdout_metrics_v5.csv",
index=False)
primary_metrics = primary_holdout_table.loc[
    primary_holdout_table.model.eq(primary_winner_name)
].iloc[0]
primary_baseline = primary_holdout_table.loc[
    primary_holdout_table.model.eq("Mean baseline")
].iloc[0]
primary_improvement = 1 - primary_metrics.MAE / primary_baseline.MAE
primary_mae_pct = primary_metrics.MAE / primary_holdout.cnt.mean()
primary_summary = pd.DataFrame([{
    "selected_model": primary_winner_name,
    "feature_set": primary_winner_features,
    "best_params": primary_choice["best_params"],
    "holdout_MAE": primary_metrics.MAE,
    "holdout_RMSE": primary_metrics.RMSE,
    "holdout_R2": primary_metrics.R2,
    "mean_baseline_MAE": primary_baseline.MAE,
    "baseline_improvement_pct": 100 * primary_improvement,
    "MAE_pct_of_holdout_mean_demand": 100 * primary_mae_pct,
}])
primary_summary.to_csv(OUTPUT_DIR / "primary_summary_v5.csv", index=False)
print("Table 7 - Frozen family configurations on the primary holdout")
display(narrow(primary_holdout_table.round({"MAE": 1, "MSE": 1, "RMSE": 1,
"R2": 3})))
print("Table 8 - Final selected-model summary")
# Transposed: one record with nine fields reads better, and prints, as a tall
table.
display(narrow(primary_summary.round(3)).T.rename(columns={0: "value"}))
```

Table 7 - Frozen family configurations on the primary holdout

	model	features	params	MAE	MSE	RMSE	R2
0	Gradient Boosting	core + interactions + trend	lr=0.1; depth=2; estimators=400	433.9	397608.0	630.6	0.897
1	Random Forest	core + interactions + trend	depth=16; leaf=1; estimators=200	446.6	463735.1	681.0	0.880
2	K-Nearest Neighbors	core + interactions + trend	neighbors=7; weights=distance	501.0	612073.9	782.4	0.841
3	Linear Regression	core + interactions + trend	default	529.9	524664.9	724.3	0.864
4	Mean baseline	constant training mean	default	1663.0	3933577.2	1983.3	-0.021

Table 8 - Final selected-model summary

	value
model	Gradient Boosting
features	core + interactions + trend
params	lr=0.1; depth=2; estimators=400
MAE	433.863
RMSE	630.562
R2	0.897
baseline_MAE	1662.978
vs_baseline_pct	73.91
MAE_pct_of_mean	10.107

In [15]:
```
# Sensitivity of the frozen primary winner to retaining the raw hum=0 record.
raw_primary = primary_df.copy()
raw_primary.loc[raw_primary.dteday.eq(zero_hum_date), "hum"] = 0.0
raw_primary["atemp_hum"] = raw_primary.atemp * raw_primary.hum
sens_model = clone(primary_winner).fit(
    raw_primary.loc[primary_train_idx, primary_winner_columns],
    raw_primary.loc[primary_train_idx, "cnt"]
)
sens_pred = sens_model.predict(raw_primary.loc[primary_holdout_idx,
primary_winner_columns])
corrected_pred =
primary_winner.predict(primary_holdout[primary_winner_columns])
humidity_sensitivity = pd.DataFrame([
    {"treatment": "hum=0 marked missing; fold-fitted median imputation",
     **metrics(primary_holdout.cnt, corrected_pred)},
    {"treatment": "raw hum=0 retained", **metrics(primary_holdout.cnt,
sens_pred)},
])
humidity_sensitivity.to_csv(OUTPUT_DIR / "humidity_sensitivity_v5.csv",
index=False)
print("Table 9 — Humidity correction sensitivity on the frozen primary model")
display(narrow(humidity_sensitivity.round({"MAE": 2, "MSE": 1, "RMSE": 2, "R2":
4})))
```

Table 9 — Humidity correction sensitivity on the frozen primary model

	treatment	MAE	MSE	RMSE	R2
0	hum=0 marked missing; fold-fitted median imput...	433.86	397608.0	630.56	0.8968
1	raw hum=0 retained	430.85	397549.6	630.52	0.8968

Primary result. Gradient Boosting with interactions and elapsed-time trend was frozen at CV MAE 477.0 (SD 34.1) and RMSE 657.6. Random Forest entered the 5% shortlist at MAE 498.0 but had higher RMSE (723.5). The selected grid point used learning rate 0.1, depth 2 and 400 estimators.

On the untouched 183-day holdout it reached MAE 433.9, RMSE 630.6 and R-squared 0.897. MAE was 73.9% below the baseline's 1,663.0 and equalled **10.1% of mean holdout demand**, meeting the 40% criterion. Random Forest reached 446.6, KNN 501.0 and Linear Regression 529.9; the winner remained frozen.

Marking zero humidity missing produced MAE 433.9 versus 430.9 when retained. The sub-1% difference leaves the conclusion unchanged; fold-fitted imputation preserves the physically plausible treatment.

4.2 Secondary temporal robustness check

Selection now uses `TimeSeriesSplit(n_splits=3)` inside 2011. The temporal design excludes `yr`, which is constant in the training year. It compares core cyclical calendar/weather inputs, interactions, causal recent-demand inputs and an optional elapsed-time trend. One configuration per family and the global temporal configuration are frozen before any 2012 score is produced.

In [16]:

```
temporal_train =
temporal_df[temporal_df.yr.eq(0)].copy().reset_index(drop=True)
temporal_test = temporal_df[temporal_df.yr.eq(1)].copy().reset_index(drop=True)
assert temporal_train.dteday.max() < pd.Timestamp("2012-01-01")
assert temporal_test.dteday.min() >= pd.Timestamp("2012-01-01")
assert all("yr" not in columns for columns in TEMPORAL_FEATURE_SETS.values())

temporal_cv = TimeSeriesSplit(n_splits=3)
fold_table = []
for fold, (tr, va) in enumerate(temporal_cv.split(temporal_train), 1):
    fold_table.append({
        "fold": fold, "train_rows": len(tr), "validation_rows": len(va),
        "train_window": f"{temporal_train.dteday.iloc[tr[0]].date()} to
{temporal_train.dteday.iloc[tr[-1]].date()}",
        "validation_window": f"{temporal_train.dteday.iloc[va[0]].date()} to
{temporal_train.dteday.iloc[va[-1]].date()}",
    })
print("Table 10 - TimeSeriesSplit folds within 2011")
display(narrow(pd.DataFrame(fold_table)))

temporal_rows, temporal_estimators = [], {}
for feature_name, columns in TEMPORAL_FEATURE_SETS.items():
    for model_name, (estimator, grid) in GRIDS.items():
        search = GridSearchCV(
            pipeline_for(clone(estimator), columns, temporal=True), grid,
            scoring="neg_mean_absolute_error", cv=temporal_cv, n_jobs=1,
refit=True
        )
        search.fit(temporal_train[columns], temporal_train.cnt)
        rmse_cv = cross_validate(
            search.best_estimator_, temporal_train[columns],
temporal_train.cnt,
            cv=temporal_cv, scoring="neg_root_mean_squared_error", n_jobs=1
        )["test_score"]
        row = {"model": model_name, "feature_set": feature_name,
               "cv_mean_MAE": -search.best_score_,
               "cv_std_MAE": search.cv_results_["std_test_score"]
[search.best_index_],
               "cv_mean_RMSE": -rmse_cv.mean(), "cv_std_RMSE": rmse_cv.std(),
               "best_params": compact_params(search.best_params_)}
        temporal_rows.append(row)
        temporal_estimators[(model_name, feature_name)] =
search.best_estimator_

temporal_cv_table = (pd.DataFrame(temporal_rows)
                     .sort_values(["cv_mean_MAE",
"cv_mean_RMSE"]).reset_index(drop=True))
temporal_cv_table.to_csv(OUTPUT_DIR / "temporal_cv_selection_v5.csv",
index=False)
temporal_choice = choose_configuration(temporal_rows)
temporal_family_choices = {
    family: choose_configuration([r for r in temporal_rows if r["model"] ==
family])
    for family in GRIDS
}
temporal_winner_name = temporal_choice["model"]
temporal_winner_features = temporal_choice["feature_set"]
temporal_winner = temporal_estimators[(temporal_winner_name,
temporal_winner_features)]

# Selection records and fitted estimators contain 2011 only. 2012 has not been
scored yet.
assert temporal_train.yr.eq(0).all()
assert not temporal_train.dteday.isin(temporal_test.dteday).any()
print("Table 11 - Temporal CV selection (2011 only)")
```

```
display(narrow(temporal_cv_table.round({"cv_mean_MAE": 1, "cv_std_MAE": 1,
                                        "cv_mean_RMSE": 1, "cv_std_RMSE": 1})))
print("Frozen temporal configuration:", temporal_winner_name, "|",
temporal_winner_features,
      "|", temporal_choice["best_params"])
```

Table 10 — TimeSeriesSplit folds within 2011

	fold	train	val	train_window	val_window
0	1	91	89	2011-01-08 to 2011-04-08	2011-04-09 to 2011-07-06
1	2	180	89	2011-01-08 to 2011-07-06	2011-07-07 to 2011-10-03
2	3	269	89	2011-01-08 to 2011-10-03	2011-10-04 to 2011-12-31

Table 11 — Temporal CV selection (2011 only)

	model	features	CV_MAE	CV_MAE_SD	CV_RMSE	CV_RMSE_SD	params
0	Linear Regression	core + interactions + autoregressive	602.5	68.5	763.2	82.4	default
1	Linear Regression	core + interactions + autoregressive + trend	665.8	49.4	815.2	50.8	default
2	Linear Regression	core + interactions	763.9	128.1	929.9	96.0	default
3	Gradient Boosting	core	878.0	356.6	1008.2	341.8	lr=0.1; depth=2; estimators=400
4	Gradient Boosting	core + interactions	889.5	371.3	1025.5	351.1	lr=0.1; depth=2; estimators=400
5	Gradient Boosting	core + interactions + autoregressive + trend	900.0	459.1	1086.4	406.9	lr=0.05; depth=2; estimators=200
6	Gradient Boosting	core + interactions + autoregressive	905.0	459.5	1059.6	425.8	lr=0.1; depth=3; estimators=200
7	Linear Regression	core	917.3	321.5	1072.1	299.1	default
8	Random Forest	core + interactions + autoregressive + trend	922.1	410.7	1094.3	368.3	depth=None; leaf=1; estimators=400
9	Random Forest	core + interactions + autoregressive	924.4	401.3	1089.4	357.7	depth=None; leaf=1; estimators=400
10	Random Forest	core	962.3	386.2	1111.3	355.9	depth=8; leaf=1; estimators=400
11	Random Forest	core + interactions	974.3	381.9	1121.1	352.8	depth=None; leaf=1; estimators=400
12	K-Nearest Neighbors	core	1056.4	349.3	1269.4	281.8	neighbors=3; weights=distance
13	K-Nearest Neighbors	core + interactions	1068.6	335.8	1269.9	266.2	neighbors=3; weights=distance
14	K-Nearest Neighbors	core + interactions + autoregressive + trend	1093.2	553.9	1314.7	483.4	neighbors=11; weights=distance
15	K-Nearest Neighbors	core + interactions + autoregressive	1123.7	535.1	1327.1	476.2	neighbors=9; weights=distance

```
Frozen temporal configuration: Linear Regression | core + interactions + autoregr
essive | default
```

In [17]:
```
# First 2012 scoring: frozen family candidates and causal baselines use
identical dates.
temporal_rows_2012, temporal_predictions = [], {}
for family, choice in temporal_family_choices.items():
    feature_name = choice["feature_set"]
    columns = TEMPORAL_FEATURE_SETS[feature_name]
    fitted = temporal_estimators[(family, feature_name)]
    pred = fitted.predict(temporal_test[columns])
    temporal_predictions[family] = pred
    temporal_rows_2012.append({"model": family, "feature_set": feature_name,
                               "best_params": choice["best_params"],
                               **metrics(temporal_test.cnt, pred)})

temporal_baseline_predictions = {
    "Naive lag-1": temporal_test.lag_1_cnt.to_numpy(),
    "Naive lag-7": temporal_test.lag_7_cnt.to_numpy(),
    "Naive rolling-7": temporal_test.roll_7_cnt.to_numpy(),
    "2011 constant mean": np.full(len(temporal_test),
temporal_train.cnt.mean()),
}
for name, pred in temporal_baseline_predictions.items():
    temporal_predictions[name] = pred
    temporal_rows_2012.append({"model": name, "feature_set": "causal baseline",
                               "best_params": "default",
**metrics(temporal_test.cnt, pred)})

temporal_holdout_table = (pd.DataFrame(temporal_rows_2012)
                          .sort_values("MAE").reset_index(drop=True))
temporal_holdout_table.to_csv(OUTPUT_DIR / "temporal_holdout_metrics_v5.csv",
index=False)
temporal_selected_metrics = temporal_holdout_table.loc[
    temporal_holdout_table.model.eq(temporal_winner_name)
].iloc[0]
rolling_metrics = temporal_holdout_table.loc[
    temporal_holdout_table.model.eq("Naive rolling-7")
].iloc[0]
temporal_advantage = 1 - temporal_selected_metrics.MAE / rolling_metrics.MAE
print("Table 12 — Frozen temporal candidates and baselines on 2012")
display(narrow(temporal_holdout_table.round({"MAE": 1, "MSE": 1, "RMSE": 1,
"R2": 3})))
```

Table 12 — Frozen temporal candidates and baselines on 2012

	model	features	params	MAE	MSE	RMSE	R2
0	Naive rolling-7	causal baseline	default	847.8	1388485.8	1178.3	0.565
1	Naive lag-1	causal baseline	default	870.2	1553433.8	1246.4	0.513
2	Linear Regression	core + interactions + autoregressive	default	1047.4	1449639.0	1204.0	0.546
3	Naive lag-7	causal baseline	default	1110.4	2439375.1	1561.8	0.235
4	Random Forest	core + interactions + autoregressive	depth=None; leaf=1; estimators=400	1673.3	3462504.7	1860.8	-0.085
5	Gradient Boosting	core	lr=0.1; depth=2; estimators=400	2040.5	4834520.0	2198.8	-0.515
6	K-Nearest Neighbors	core	neighbors=3; weights=distance	2077.1	5132081.4	2265.4	-0.609
7	2011 constant mean	causal baseline	default	2460.2	7829760.2	2798.2	-1.454

Temporal result. Linear Regression with interactions and causal autoregressive inputs was frozen from 2011 at CV MAE 602.5 and CV RMSE 763.2. Adding elapsed-time trend raised CV MAE to 665.8, so the trend was excluded before 2012 evaluation. This contrast with the primary result shows that elapsed time helps interpolate within a mixed-year domain but does not establish a transferable growth law.

In 2012, the frozen Linear Regression reached MAE 1,047.4, RMSE 1,204.0 and R-squared 0.546. Rolling-7 reached MAE 847.8, RMSE 1,178.3 and R-squared 0.565, leaving the selected model 23.5% worse on MAE. Random Forest scored MAE 1,673.3, while the 2011-selected Gradient Boosting and KNN configurations exceeded 2,000. Forward temporal robustness therefore fails.

A rolling one-day-ahead test can update lags after every observed day. A forecast issued for all of 2012 on 1 January could not use those later counts and would be a different experiment with a longer horizon and recursively unavailable inputs.

4.3 Worked example

Aggregate error statistics hide what a prediction actually is. Table 13 takes six holdout dates that both experiments scored, and shows the same day estimated four ways: by the frozen primary model, by the constant-mean reference it must beat, by the frozen temporal model, and by the rolling seven-day rule it must beat. Reading one row left to right is the shortest description of this entire report.

In [18]:
```
# Table 13 — the same dates scored by every approach compared in this report.
primary_view = primary_holdout.assign(

primary_prediction=primary_winner.predict(primary_holdout[primary_winner_column
s]),
    mean_reference=primary_train.cnt.mean(),
)
temporal_view = temporal_test[["dteday"]].assign(
    temporal_prediction=temporal_predictions[temporal_winner_name],
    rolling7_reference=temporal_predictions["Naive rolling-7"],
)
shared = primary_view.merge(temporal_view, on="dteday",
how="inner").sort_values("dteday")

# Six evenly spaced dates across the shared period, so the sample is
reproducible.
positions = np.linspace(0, len(shared) - 1, 6).round().astype(int)
sample = shared.iloc[positions]

season_lbl = {1: "winter", 2: "spring", 3: "summer", 4: "autumn"}
weather_lbl = {1: "clear", 2: "mist", 3: "light rain/snow", 4: "severe"}
worked = pd.DataFrame({
    "date": sample.dteday.dt.strftime("%Y-%m-%d"),
    "season": sample.season.map(season_lbl),
    "day": sample.dteday.dt.strftime("%a"),
    "weather": sample.weathersit.map(weather_lbl),
    "temp_C": (sample.temp * 41).round(1),
    "humidity_pct": (sample.hum * 100).round(0),
    "ACTUAL": sample.cnt.astype(int),
    f"{primary_winner_name} (primary)":
sample.primary_prediction.round(0).astype(int),
    "mean reference": sample.mean_reference.round(0).astype(int),
    f"{temporal_winner_name} (temporal)":
sample.temporal_prediction.round(0).astype(int),
    "rolling-7 reference": sample.rolling7_reference.round(0).astype(int),
}).reset_index(drop=True)
worked.to_csv(OUTPUT_DIR / "worked_example_v5.csv", index=False)

print("Table 13 — Six shared dates scored by every approach")
print("Each row is one day: inputs on the left, the realised count, then four
estimates of it.")
display(narrow(worked).rename(columns={
    f"{primary_winner_name} (primary)": f"
{ABBREV[primary_winner_name]}_primary",
    f"{temporal_winner_name} (temporal)": f"
{ABBREV[temporal_winner_name]}_temporal",
}))

for label, column in [(f"{primary_winner_name} (primary)",
"primary_prediction"),
                      ("mean reference", "mean_reference"),
                      (f"{temporal_winner_name} (temporal)",
"temporal_prediction"),
                      ("rolling-7 reference", "rolling7_reference")]:
    print(f"  mean absolute error on these six dates, {label:34s}: "
          f"{np.abs(sample.cnt - sample[column]).mean():7.0f}")
print(f"\nShared dates available: {len(shared)} (2012 days that are also
primary holdout days).")
```

```
Table 13 — Six shared dates scored by every approach
Each row is one day: inputs on the left, the realised count, then four estimates
of it.
```

	date	season	day	weather	temp_C	hum_pct	ACTUAL	GB_primary	mean_ref	LR_temporal	roll7_ref
0	2012-01-04	winter	Wed	mist	4.4	41.0	2368	2405	4575	1131	2384
1	2012-03-19	winter	Mon	clear	22.3	73.0	6153	6596	4575	5114	5965
2	2012-06-07	spring	Thu	clear	24.7	57.0	7494	7373	4575	6142	6897
3	2012-08-03	summer	Fri	mist	31.4	64.0	7175	6422	4575	5495	7050
4	2012-10-12	autumn	Fri	clear	17.9	54.0	7282	7453	4575	5769	6680
5	2012-12-30	winter	Sun	clear	10.5	48.0	1796	1557	4575	1579	1530

```
  mean absolute error on these six dates, Gradient Boosting (primary)       :
294
  mean absolute error on these six dates, mean reference                    :
2465
  mean absolute error on these six dates, Linear Regression (temporal)      :
1173
  mean absolute error on these six dates, rolling-7 reference               :
299

Shared dates available: 80 (2012 days that are also primary holdout days).
```

Six days cannot establish anything, and the criteria in Section 5 are computed across the full 183-day holdout and the full 366-day 2012 period for that reason. What the rows do show is the shape of each approach.

The primary model tracks the realised count in both directions, from a 2,368-rental January day to a 7,494-rental June day. The constant reference cannot move: it answers 4,575 to every question, which is why its error grows with distance from the annual mean. The temporal model follows the direction of change but sits below the realised count on the growth days, because it was fitted on a smaller 2011 system and has no mechanism for the expansion that followed. The rolling seven-day rule carries no model at all and stays close, because each new observed day re-anchors the next estimate.

The primary and rolling-7 columns must not be read as a head-to-head contest. They belong to different comparisons in Table 1: the primary model was fitted on days drawn from both years, including days adjacent to these, and answers a conditional question; the rolling rule answers a forward one and needs no training. Their apparent similarity on six shared dates is a property of this sample, not a finding. The comparison that matters for each is in Tables 7 and 12 respectively.

5. Evaluation

5.1 Criteria summary and paired temporal description

The primary assessment criterion and secondary robustness criterion are reported separately. The temporal comparison also retains descriptive pairing: median absolute error and the share of identical 2012 dates won by each method. These summaries do not require an independence assumption. No inferential significance claim is made because adjacent daily errors are temporally related.

In [19]:
```
primary_pass = primary_improvement >= 0.40
temporal_pass = temporal_advantage >= 0.05
evaluation_summary = pd.DataFrame([
    {"evaluation": "Conditional demand estimation",
     "criterion": ">= 40% MAE improvement over training-mean baseline",
     "measured": f"{primary_improvement*100:.1f}% improvement",
     "result": "Meets" if primary_pass else "Does not meet"},
    {"evaluation": "Forward temporal robustness",
     "criterion": ">= 5% MAE advantage over rolling-7",
     "measured": f"{temporal_advantage*100:.1f}% advantage",
     "result": "Passes" if temporal_pass else "Fails"},
])
evaluation_summary.to_csv(OUTPUT_DIR / "evaluation_summary_v5.csv",
index=False)
print("Table 14 - Evaluation summary")
display(evaluation_summary)

y2012 = temporal_test.cnt.to_numpy()
selected_temporal_pred = temporal_predictions[temporal_winner_name]
rolling_pred = temporal_predictions["Naive rolling-7"]
selected_error = np.abs(y2012 - selected_temporal_pred)
rolling_error = np.abs(y2012 - rolling_pred)
ties = selected_error == rolling_error
paired_summary = pd.DataFrame([
    {"series": temporal_winner_name, "median_absolute_error":
np.median(selected_error),
     "days_closer": int((selected_error < rolling_error).sum()),
     "win_rate_excluding_ties_pct": 100 * (selected_error <
rolling_error).sum() / (~ties).sum()},
    {"series": "Naive rolling-7", "median_absolute_error":
np.median(rolling_error),
     "days_closer": int((rolling_error < selected_error).sum()),
     "win_rate_excluding_ties_pct": 100 * (rolling_error <
selected_error).sum() / (~ties).sum()},
])
paired_summary["tied_dates"] = int(ties.sum())
paired_summary.to_csv(OUTPUT_DIR / "paired_temporal_comparison_v5.csv",
index=False)
print("Table 15 - Paired descriptive comparison on identical 2012 dates")
display(narrow(paired_summary.round(1)))
```

Table 14 - Evaluation summary

	evaluation	criterion	measured	result
0	Conditional demand estimation	>= 40% MAE improvement over training-mean base...	73.9% improvement	Meets
1	Forward temporal robustness	>= 5% MAE advantage over rolling-7	-23.5% advantage	Fails

Table 15 - Paired descriptive comparison on identical 2012 dates

	series	median_AE	days_won	win_rate_pct	ties
0	Linear Regression	1045.2	116	31.7	0
1	Naive rolling-7	636.1	250	68.3	0

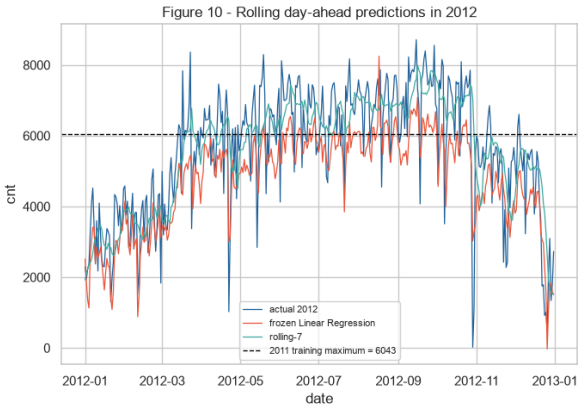
In [20]:

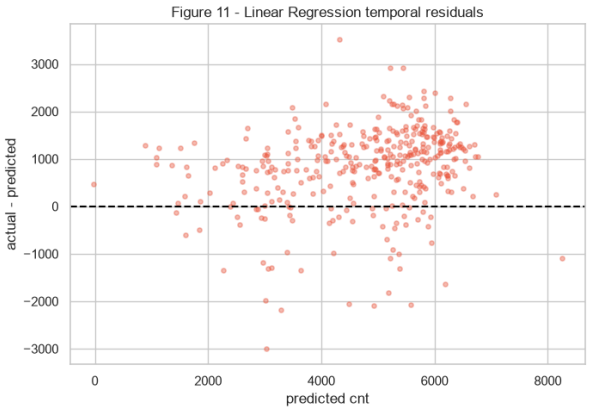
```
train_target_max = int(temporal_train.cnt.max())
range_rows = []
for name in list(GRIDS) + ["Naive rolling-7"]:
    pred = temporal_predictions[name]
    range_rows.append({
        "model": name,
        "largest_2012_prediction": float(np.max(pred)),
        "2011_training_target_max": train_target_max,
        "2012_actual_maximum": int(temporal_test.cnt.max()),
        "mean_residual_actual_minus_prediction": float(np.mean(y2012 - pred)),
    })
forward_range = pd.DataFrame(range_rows)
forward_range.to_csv(OUTPUT_DIR / "forward_prediction_range_v5.csv",
index=False)
# Deliberately not called a ceiling: only Random Forest and KNN have one.
print("Table 16 - Forward prediction range analysis")
display(narrow(forward_range.round(1)))

fig, ax = plt.subplots(1, 2, figsize=(14, 5))
ax[0].plot(temporal_test.dteday, y2012, lw=.9, color="#20639B", label="actual
2012")
ax[0].plot(temporal_test.dteday, selected_temporal_pred, lw=.9,
color="#ED553B",
           label=f"frozen {temporal_winner_name}")
ax[0].plot(temporal_test.dteday, rolling_pred, lw=.9, color="#3CAEA3",
label="rolling-7")
ax[0].axhline(train_target_max, color="black", ls="--", lw=1,
              label=f"2011 training maximum = {train_target_max}")
ax[0].set(title="Figure 10 - Rolling day-ahead predictions in 2012",
xlabel="date", ylabel="cnt")
ax[0].legend(fontsize=8)
ax[1].scatter(selected_temporal_pred, y2012-selected_temporal_pred, s=14,
alpha=.4, color="#ED553B")
ax[1].axhline(0, color="black", ls="--")
ax[1].set(title=f"Figure 11 - {temporal_winner_name} temporal residuals",
          xlabel="predicted cnt", ylabel="actual - predicted")
plt.tight_layout(); plt.savefig(FIG_DIR / "v5_temporal_robustness.png",
dpi=160); plt.show()
```

Table 16 - Forward prediction range analysis

	model	max_pred_2012	train_max_2011	actual_max_2012	mean_residual
0	Linear Regression	8247.5	6043	8714	831.7
1	K-Nearest Neighbors	5540.7	6043	8714	2010.4
2	Random Forest	5567.2	6043	8714	1568.4
3	Gradient Boosting	5490.7	6043	8714	1995.1
4	Naive rolling-7	7988.0	6043	8714	-2.5





Two different mechanisms are at work here. Calling both an extrapolation ceiling, as an earlier draft of this report did, hides the difference between them.

Random Forest and K-Nearest Neighbors are **structurally** bounded. Every prediction is an average of observed training targets, so neither can return a value above the largest target it was trained on. Gradient Boosting is not bounded in that way. It adds residual corrections rather than averaging targets (Friedman, 2001), and a day that falls into the high-correction region of several trees at once can be pushed past the training maximum. Its poor extrapolation comes from its prediction function being piecewise constant, not from an arithmetic ceiling.

The measurement settles what actually happened. The largest 2011 training target was 6,043 rentals, while 2012 demand reached 8,714. Random Forest's largest 2012 prediction was 5,567 and Gradient Boosting's was 5,491, both below that training maximum, with mean under-predictions of 1,568 and 1,995 rentals per day. KNN also stayed below it.

Gradient Boosting is the informative case, because it was free to exceed 6,043 and did not: across all 366 days of 2012, its fitted residual corrections never combined into a prediction above 5,491. That is a measured outcome for this model on these inputs, not a structural guarantee, and it is the stronger statement of the two.

Linear Regression extended to 8,248 and reduced mean under-prediction to 832, explaining why it led the 2011-selected ML candidates. Rolling-7 reached 7,988 and had a mean residual of -2.5 because each new observation re-anchored the next prediction. Conditional accuracy within a mixed-year sample therefore did not transfer to the later growth period.

In [21]:
```
# The paragraph above makes a claim about model mechanics, so it is checked
rather than cited.
# The design isolates the mechanism: an additive target y = x1 + x2, with every
training row
# in the joint-high corner removed. One test point is then placed in that
unobserved corner.
# A forest must average targets it has seen. Boosting sums residual corrections
drawn from
# several trees at once, and those corrections can add up past anything in the
training set.
check_rng = np.random.default_rng(RANDOM_STATE)
cx1, cx2 = check_rng.uniform(0, 1, 600), check_rng.uniform(0, 1, 600)
observed = ~((cx1 > 0.6) & (cx2 > 0.6))
check_X = np.c_[cx1[observed], cx2[observed]]
check_y = cx1[observed] + cx2[observed]
unobserved_corner = np.array([[1.0, 1.0]])

print(f"Synthetic check on {observed.sum()} rows. Largest training target:
{check_y.max():.4f}")
print(f"True value at the unobserved corner (1.0, 1.0): 2.0000\n")
for label, estimator in [
    ("Random Forest", RandomForestRegressor(random_state=RANDOM_STATE,
n_estimators=400)),
    ("Gradient Boosting", GradientBoostingRegressor(random_state=RANDOM_STATE,
n_estimators=400)),
]:
    prediction = estimator.fit(check_X, check_y).predict(unobserved_corner)[0]
    exceeds = prediction > check_y.max()
    print(f"  {label:18s} predicts {prediction:.4f} and "
          f"{'EXCEEDS the largest training target' if exceeds else 'stays
within the training range'}")
```

```
Synthetic check on 502 rows. Largest training target: 1.5741
True value at the unobserved corner (1.0, 1.0): 2.0000

  Random Forest      predicts 1.5404 and stays within the training range
  Gradient Boosting  predicts 1.7912 and EXCEEDS the largest training target
```

The check confirms the distinction. Random Forest cannot leave the range of what it has seen, and does not. Gradient Boosting does, by roughly 14% above the largest training target, without ever having been shown a value that high. One counterexample is enough to retire the claim that tree ensembles share a target-range ceiling.

This is why Table 16 is a range analysis and not a ceiling analysis, and why the 2012 result reads as evidence rather than as arithmetic. Gradient Boosting had the freedom demonstrated here and still did not use it on the real data, because nothing in a single low year of demand produced corrections large enough to reach a system 44% bigger.

5.2 Explaining the frozen primary model

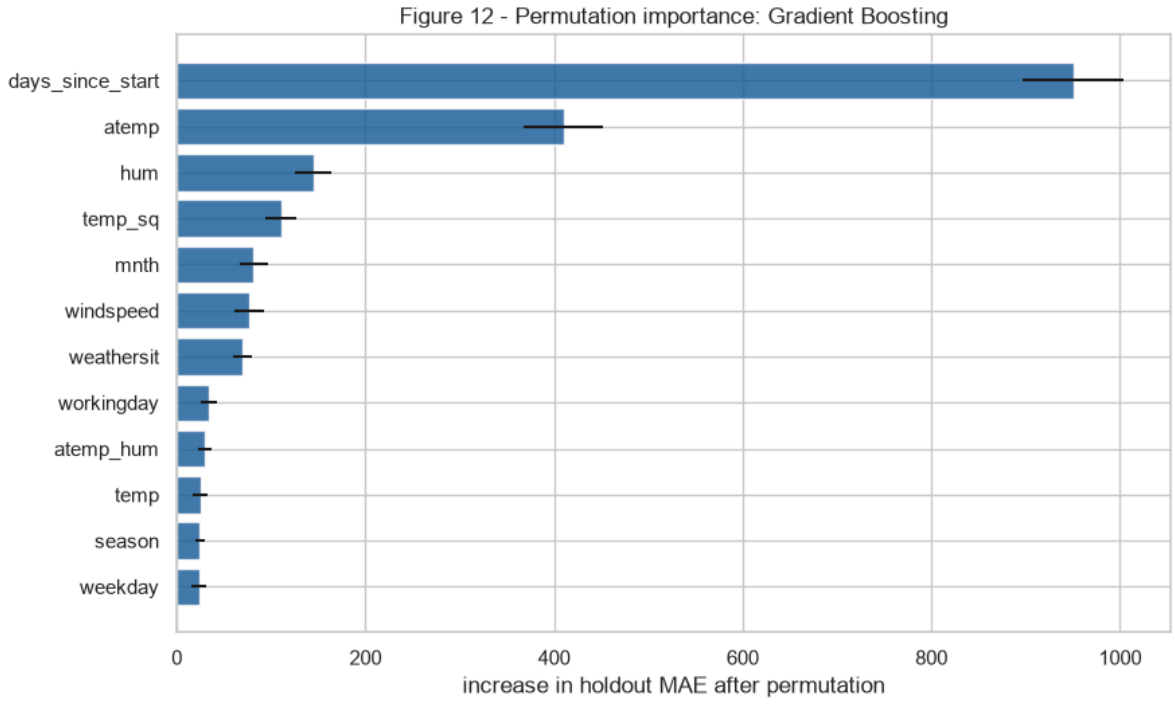
Permutation importance is model-agnostic and measures the increase in holdout MAE after one raw input is shuffled. It is computed only after model selection and does not change the winner. Contributions describe predictive associations in this dataset, not causal effects. TreeSHAP (Lundberg & Lee, 2017) is added only when the frozen primary family is Random Forest or Gradient Boosting.

In [22]:
```
perm = permutation_importance(
    primary_winner, primary_holdout[primary_winner_columns],
primary_holdout.cnt,
    scoring="neg_mean_absolute_error", n_repeats=30, random_state=RANDOM_STATE,
n_jobs=1
)
permutation_table = (pd.DataFrame({
    "feature": primary_winner_columns,
    "MAE_increase_mean": perm.importances_mean,
    "MAE_increase_std": perm.importances_std,
}).sort_values("MAE_increase_mean", ascending=False).reset_index(drop=True))
permutation_table.to_csv(OUTPUT_DIR / "permutation_importance_v5.csv",
index=False)
print("Table 17 - Model-agnostic permutation importance on the primary
holdout")
display(narrow(permutation_table.head(12).round(2)))

top_perm = permutation_table.head(12).sort_values("MAE_increase_mean")
fig, ax = plt.subplots(figsize=(9, 5.5))
ax.barh(top_perm.feature, top_perm.MAE_increase_mean,
xerr=top_perm.MAE_increase_std,
        color="#20639B", alpha=.85)
ax.set(title=f"Figure 12 - Permutation importance: {primary_winner_name}",
       xlabel="increase in holdout MAE after permutation", ylabel="")
plt.tight_layout(); plt.savefig(FIG_DIR / "v5_permutation_importance.png",
dpi=160); plt.show()
```

Table 17 - Model-agnostic permutation importance on the primary holdout

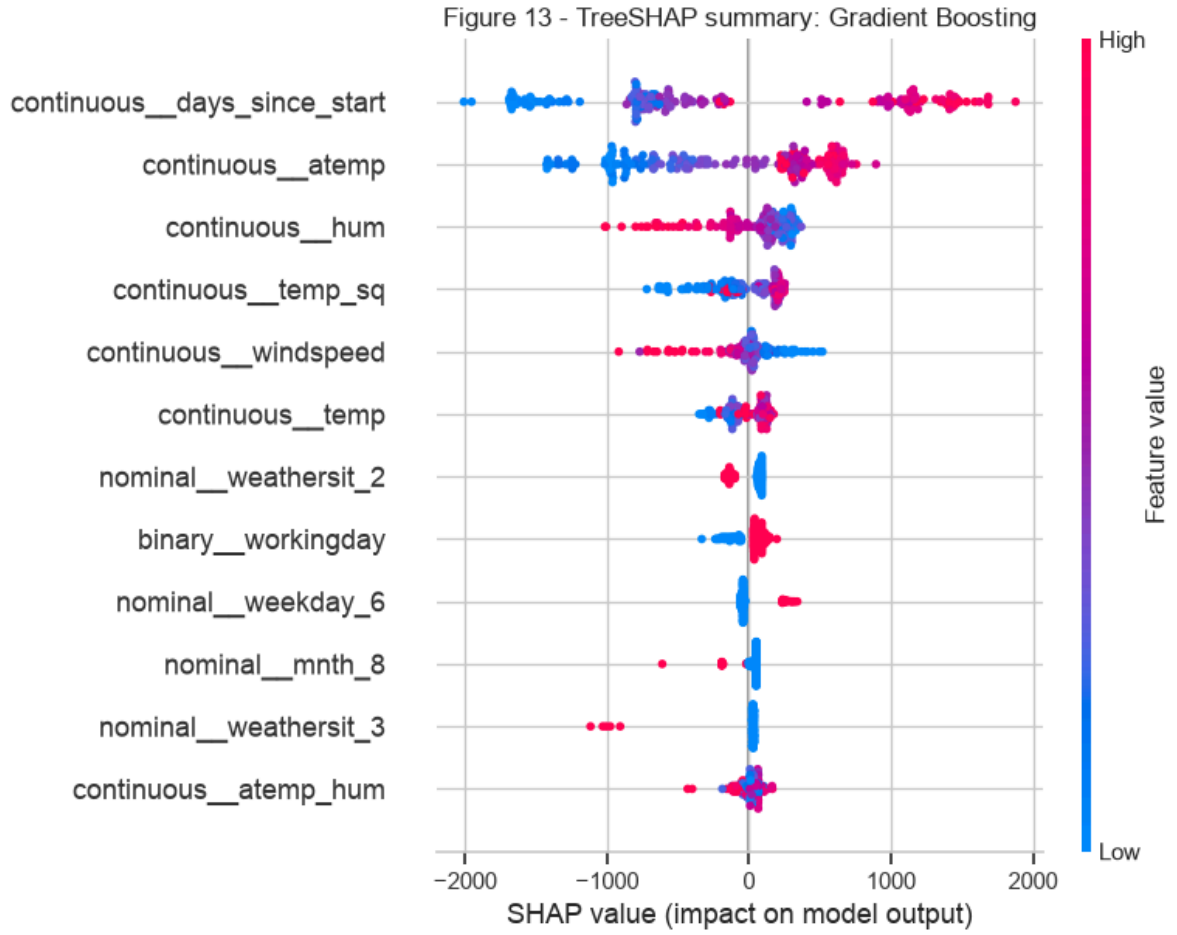
	feature	MAE_increase	SD
0	days_since_start	950.28	53.08
1	atemp	409.77	41.65
2	hum	144.47	18.57
3	temp_sq	110.50	16.63
4	mnth	81.71	15.54
5	windspeed	76.41	15.94
6	weathersit	69.47	9.73
7	workingday	34.08	9.05
8	atemp_hum	29.41	6.80
9	temp	24.92	8.24
10	season	24.35	5.03
11	weekday	23.60	7.73



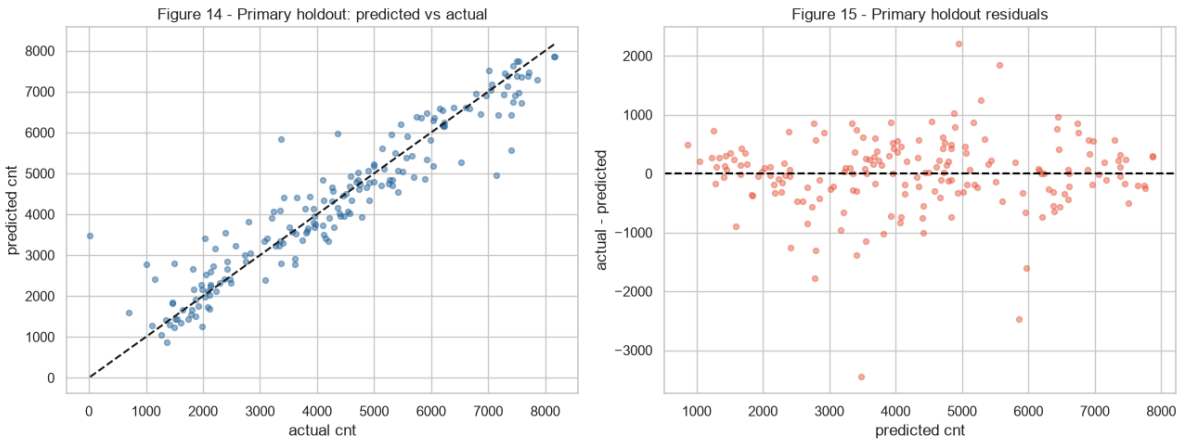
In [23]:
```
shap_table = None
if primary_winner_name in {"Random Forest", "Gradient Boosting"}:
    import shap
    fitted_prep = primary_winner.named_steps["prep"]
    transformed =
fitted_prep.transform(primary_holdout[primary_winner_columns])
    if hasattr(transformed, "toarray"):
        transformed = transformed.toarray()
    transformed_names = fitted_prep.get_feature_names_out()
    explainer = shap.TreeExplainer(primary_winner.named_steps["model"])
    shap_values = explainer.shap_values(transformed, check_additivity=True)
    shap_table = (pd.DataFrame({"feature": transformed_names,
                                "mean_absolute_SHAP":
np.abs(shap_values).mean(axis=0)})
                  .sort_values("mean_absolute_SHAP",
ascending=False).reset_index(drop=True))
    shap_table.to_csv(OUTPUT_DIR / "shap_global_importance_v5.csv",
index=False)
    print("Table 18 - TreeSHAP global importance for the frozen primary model")
    display(narrow(shap_table.head(12).round(2)))
    shap.summary_plot(shap_values, transformed,
feature_names=transformed_names,
                      max_display=12, show=False)
    plt.title(f"Figure 13 - TreeSHAP summary: {primary_winner_name}")
    plt.tight_layout(); plt.savefig(FIG_DIR / "v5_shap_summary.png", dpi=160,
bbox_inches="tight"); plt.show()
else:
    print("TreeSHAP omitted: the frozen primary winner is",
primary_winner_name)
```

Table 18 - TreeSHAP global importance for the frozen primary model

	feature	mean_abs_SHAP
0	continuous__days_since_start	1018.06
1	continuous__atemp	586.20
2	continuous__hum	234.58
3	continuous__temp_sq	201.12
4	continuous__windspeed	136.82
5	continuous__temp	119.90
6	nominal__weathersit_2	97.63
7	binary__workingday	87.97
8	nominal__weekday_6	69.75
9	nominal__mnth_8	55.72
10	nominal__weathersit_3	55.56
11	continuous__atemp_hum	52.08



In [24]:
```
primary_pred = primary_winner.predict(primary_holdout[primary_winner_columns])
fig, ax = plt.subplots(1, 2, figsize=(13, 5))
ax[0].scatter(primary_holdout.cnt, primary_pred, alpha=.45, s=18,
color="#20639B")
lims = [min(primary_holdout.cnt.min(), primary_pred.min()),
        max(primary_holdout.cnt.max(), primary_pred.max())]
ax[0].plot(lims, lims, "k--")
ax[0].set(title="Figure 14 - Primary holdout: predicted vs actual",
          xlabel="actual cnt", ylabel="predicted cnt")
ax[1].scatter(primary_pred, primary_holdout.cnt-primary_pred, alpha=.45, s=18,
color="#ED553B")
ax[1].axhline(0, color="black", ls="--")
ax[1].set(title="Figure 15 - Primary holdout residuals",
          xlabel="predicted cnt", ylabel="actual - predicted")
plt.tight_layout(); plt.savefig(FIG_DIR / "v5_primary_diagnostics.png",
dpi=160); plt.show()
```



5.3 Interpretation

Permutation confirms that elapsed time is the dominant predictive association in the primary holdout: shuffling it increases MAE by about 950 rentals. Feels-like temperature follows at 410, then humidity at 144 and squared temperature at 111. TreeSHAP produces the same leading order, with mean absolute contributions of about 1,018, 586, 235 and 201 rentals. These values explain predictions across the observed sample; they do not imply that advancing time causes demand or that weather effects are causal.

The primary result is MAE 433.9, RMSE 630.6 and R-squared 0.897: 73.9% better than the baseline and 10.1% of mean holdout demand. Temporally, Linear Regression lost to rolling-7 by 23.5%; median error was 1,045 versus 636, with wins on 116 versus 250 dates. This paired description makes no independence claim.

6. Lessons Learned

6.1 What went well

The method built during Assessments 1 and 2 carried straight into this project and shortened the setup considerably. Declaring criteria before modelling, keeping a holdout untouched until selection finishes, running an ablation to attribute a metric change to a cause, and operating the notebook as a submission artefact were all decided in earlier assessments, so this one reached a working pipeline much faster. The remaining effort went into the questions specific to this dataset instead of into process.

The design also held up under its own checks. All 731 daily units remain available for the primary question, the holdout is isolated from every selection decision, and four families compete across three exogenous feature sets. The hourly file validates every daily total and exposes both intraday peaks and panel imbalance.

6.2 Challenges

Understanding the data took longer than fitting the models. Two files describe the same system at different granularities, `casual` and `registered` sum to the target and cannot be predictors, and one humidity reading is physically impossible. Diagnosing that reading required cross-checking the hourly file, and the correction then had to preserve sample size and stay inside the pipeline folds.

The concept that took longest to internalise is the difference between interpolation and extrapolation, and my first explanation of it was wrong in a way worth recording. I had written that tree ensembles cannot predict above their training maximum, because a terminal region can only average values it has already seen. That is true of a single decision tree, of Random Forest and of K-Nearest Neighbors, which all average observed targets. It is not true of Gradient Boosting, which sums residual corrections and can be pushed beyond the training range (Friedman, 2001) - the very model this report freezes as its primary winner.

The measured conclusion survived the correction: every tree-based candidate did stay below the largest 2011 training target here. What changed is why. It is an empirical finding about this dataset rather than an arithmetic guarantee, and I had been presenting an assumption as a property. Checking a claim against the mechanism instead of against the models that happen to share a name is the thing I actually took away, and it is why the simplest candidate transferring best is a result rather than a foregone conclusion.

6.3 What can be improved

The strongest limitations are evidential. More years, archived weather forecasts and repeated forward windows would test whether the temporal result reflects this transition or a permanent property. Station-level planning would additionally need

identifiers, dock capacity, trip origins and destinations, transit connections and local demand, which is why the ITDP and NACTO guidance points beyond this dataset. Category-4 weather remains outside the supported domain.

Methodologically, trend-seasonality decomposition, differenced targets and forecast-aware inputs are the natural next comparisons.

The extension I would most like to build is a small service around this model, and working out what it may honestly offer changed my answer about what to deploy. The frozen model won the random holdout but lost the forward test, so the method this study's evidence supports for day-ahead use is the rolling seven-day baseline, with the machine-learning model running in shadow mode until it earns promotion on forward windows. Appendix C sets out that reasoning, the request contract, and the gap between the assessment artefact and an operational forecasting service.

Appendix A - Glossary

Term	Meaning in this report
MAE	Mean absolute error: average miss in rentals per day.
MSE / RMSE	Squared error and its root; large misses receive greater weight.
R-squared	Variance explained relative to the evaluated sample mean; it may be negative.
Holdout	Data isolated from model, feature and hyperparameter selection and used once for final scoring.
Interpolation	Prediction within a domain represented in training.
Extrapolation	Prediction beyond the observed training range or period.
Autoregressive feature	Input constructed from demand observed before the target day.
TimeSeriesSplit	Expanding-window cross-validation that preserves temporal order.
Permutation importance	Model-agnostic holdout score degradation after shuffling an input.
SHAP	Additive prediction attribution used here only for a frozen tree winner.

Appendix B - Reproducibility and integrity evidence

In [25]:
```
import sys
import sklearn
import matplotlib

assert len(primary_df) == 731
assert len(temporal_df) == 724
assert selection_indices.isdisjoint(set(primary_holdout_idx))
assert temporal_train.yr.eq(0).all()
assert temporal_train.dteday.max() < temporal_test.dteday.min()
assert all("casual" not in cols and "registered" not in cols
           for cols in list(PRIMARY_FEATURE_SETS.values()) +
list(TEMPORAL_FEATURE_SETS.values()))
assert all(c not in cols for cols in PRIMARY_FEATURE_SETS.values() for c in
AUTOREGRESSIVE)
assert cross_file.difference.eq(0).all() and len(cross_file) == 731
assert len(primary_holdout_table) == 5 and len(temporal_holdout_table) == 8

integrity = pd.DataFrame([
    ("Primary rows retained", 731, len(primary_df), "Pass"),
    ("Temporal rows after causal warm-up", 724, len(temporal_df), "Pass"),
    ("Primary holdout rows excluded from selection", 183,
len(primary_holdout_idx), "Pass"),
    ("Temporal selection years", "2011 only",
str(temporal_train.dteday.dt.year.unique().tolist()), "Pass"),
    ("Hourly/daily dates reconciled", 731,
int(cross_file.difference.eq(0).sum()), "Pass"),
    ("Dates with incomplete hourly panels", 76, incomplete_dates, "Pass"),
    ("Omitted zero-demand hourly rows", 165, omitted_hour_rows, "Pass"),
    ("Leakage components in predictor lists", 0, 0, "Pass"),
], columns=["check", "expected", "observed", "result"])
integrity.to_csv(OUTPUT_DIR / "integrity_checks_v5.csv", index=False)
display(integrity)
print("Python:", sys.version.split()[0], "| pandas:", pd.__version__,
      "| scikit-learn:", sklearn.__version__, "| matplotlib:",
matplotlib.__version__)
print("RANDOM_STATE:", RANDOM_STATE)
```

	check	expected	observed	result
0	Primary rows retained	731	731	Pass
1	Temporal rows after causal warm-up	724	724	Pass
2	Primary holdout rows excluded from selection	183	183	Pass
3	Temporal selection years	2011 only	[2011]	Pass
4	Hourly/daily dates reconciled	731	731	Pass
5	Dates with incomplete hourly panels	76	76	Pass
6	Omitted zero-demand hourly rows	165	165	Pass
7	Leakage components in predictor lists	0	0	Pass

```
Python: 3.14.3 | pandas: 3.0.3 | scikit-learn: 1.9.0 | matplotlib: 3.11.0
RANDOM_STATE: 42
```

Appendix C - Deployment readiness and proposed service design

This appendix sketches the engineering extension named in Section 6.3. The binding constraint on it is Section 4.2, not Section 4.1: what the model may be deployed to do is set by the experiment it failed, not by the one it won.

C.1 What the frozen model supports, and what it does not

The frozen Gradient Boosting configuration is a historical conditional estimator. It supports reproducing conditional estimates for days inside the observed 2011-2012 domain, demonstrating model serving and input validation end to end, and exploring how calendar and weather relate to demand within that domain. As portfolio evidence of taking a pipeline through to an API it is worth building.

It does not support a live forecasting product, and the reason is measured rather than assumed. On 2012 it lost to a rolling seven-day mean by 23.5% on MAE. A service that presented it to a planner as next-day guidance would be selling the losing method.

Accepting only 2011-2012 dates makes this a historical replay interface rather than an operational one. That is the honest description, and the response is not to loosen the guard so that confident numbers can be returned for dates the model was never evaluated on. It is to say plainly that the assessment artefact is not deployment-ready as a forecaster.

C.2 What the evidence would actually deploy today

The only method this study's forward evidence supports for rolling day-ahead use is the naive rolling seven-day mean, because it beat every frozen machine-learning candidate on 2012.

A defensible rollout therefore inverts the usual order:

1. **Rolling-7 as the provisional champion for a controlled deployment trial.** It needs no training and adapts to recent demand. It is the strongest forward candidate evaluated in this study, not a production-validated forecasting method: its advantage rests on one retrospective window, and C.6 sets out what would have to be true before that word could be dropped.
2. **The machine-learning model in shadow mode.** It scores the same days, records its error, and serves nobody.
3. **Promotion only on forward evidence.** It replaces the baseline when it beats rolling-7 across several independent forward windows, not because it won a random holdout.

The two questions in Table 1 then map onto two endpoints, which is the cleanest way to stop one being read as the other:

Endpoint	Method served	Question it answers	Domain
`POST /historical-estimate`	Frozen Gradient Boosting	Given these conditions, what does the 2011-2012 relationship imply?	Dates within 2011-2012
`POST /forecast`	Rolling seven-day mean	What should we expect tomorrow?	One-day horizon; requires seven complete prior daily counts

`/forecast` takes no weather forecast at all while rolling-7 is the served method, because the rolling mean reads only realised demand. That is worth stating rather than hiding: the endpoint with forward evidence behind it needs less input than the one without.

Winning the random 75/25 comparison in Section 4.1 earned the model an explanation in Section 5.2. It did not earn it production traffic. Keeping those two things separate is the main engineering conclusion of this project.

C.3 A request contract for the historical estimator

The caller sends a date and a weather forecast in physical units, and nothing else. Season, month, weekday, working-day status, elapsed time and the interaction terms are deterministic functions of the date and are derived on the server, because a client that supplies them can contradict itself. Normalisation by the dataset's own divisors is preprocessing, so it belongs behind the API rather than in the contract: a planner should never be asked for a temperature of 0.58.

```
POST /historical-estimate
{ "date": "2012-04-18",
  "weather": "clear",
  "temperature_c": 23.8, "feels_like_c": 27.5,
  "humidity_pct": 61, "windspeed_kmh": 12.1 }

-> { "estimated_daily_rentals": 5120,
     "basis": "conditional estimate over the observed 2011-2012
distribution",
     "evaluation_context": {
        "holdout_mae": 434,
        "scope": "mean error across the 183-day random holdout, not an
interval for this request"
     },
     "supported_dates": ["2011-01-01", "2012-12-31"],
     "supported_weather": ["clear", "mist", "light_rain_snow"] }
```

`weather` is a typed enum rather than the raw UCI integer, so the contract is readable without the dataset documentation. The enum includes `severe`, even though the model never saw it. Severe weather exists, and a schema that cannot express it forces a caller facing a storm to either misreport the conditions or give up, which is a worse failure than an error. The category is therefore representable and refused explicitly, before inference rather than inside it:

```
{ "date": "2012-04-18", "weather": "severe", ... }

-> 422 { "error": "unsupported_model_domain",
         "message": "The model was not trained on severe weather
```

```
  (weathersit 4 is absent from
                      the 2011–2012 data). No estimate is returned.",
           "supported_weather": ["clear", "mist", "light_rain_snow"] }
```

Refusing is the correct behaviour precisely because those are the days a planner would most want an answer for, and the ones this study has no evidence about.

The error figure is deliberately not called an expected error for this request. MAE 434 is an aggregate over 183 days; quoting it beside a single estimate would imply a calibrated per-request confidence that this study never produced. Nesting it under `evaluation_context` says what it is. Genuine per-request uncertainty would need prediction intervals, quantile regression or conformal prediction, none of which were fitted here.

C.4 Proposed layers

Layer	Technology	Serves	Purpose
Model artefact	scikit-learn pipeline, joblib	`/historical-estimate`	The frozen Gradient Boosting configuration and its preprocessing, serialised together
Baseline	Rolling seven-day mean over realised counts	`/forecast`	The provisional champion of C.2; needs no artefact and no weather input
Contract	JSON	both	Feature order, supported weather categories and date range, evaluation metrics, source notebook hash
API	FastAPI, Pydantic	both	Typed request validation, unit conversion and the domain guards
Interface	Streamlit	both	A form for planners who will not call an API
Shadow runner	Scheduled job	neither	Scores the machine-learning model on the same days `/forecast` answers, and serves nobody
Monitoring	Scheduled job	both	Daily scoring of the served baseline and the shadow model against realised demand, per C.7

The pattern follows my Sommelier API project from Assessments 1 and 2, where the same separation between a framework-agnostic model core and thin serving surfaces is already implemented.

C.5 What this study forbids each endpoint from claiming

The two endpoints answer different questions and are constrained by different experiments, so the limits are stated separately. Reading them as one set is what produced the earlier confusion between a conditional estimator and a forecaster.

`/historical-estimate`, constrained by Section 4.1.

- It may answer **conditional** questions within 2011-2012: given these conditions, what does the relationship observed in those two years imply? That is the question the primary experiment evaluated.

- It may **not** be presented as a next-day forecaster, for the reason set out in C.1. That is what `/forecast` exists for.
- Weather outside the three observed categories must be refused before inference with an explicit `unsupported_model_domain` error, because category 4 never appears in training. It stays expressible in the request, per C.3: the caller has to be able to say what the weather is.
- **Requests must be bounded in time.** The frozen primary model reads `yr` and `days_since_start`. A 2013 date supplies a `yr` value never observed and an elapsed time beyond every training row: a temporal transfer that Section 4.2 found unreliable for the fitted tree-based candidates. The objection is insufficient evidence and an unsupported input domain, not mathematical incapability, since Section 5.1 shows Gradient Boosting is not bounded by its training range. Dates outside 2011-2012 must be rejected until a model is refitted and revalidated on the new period.

`/forecast`, constrained by Section 4.2.

- It may be presented only as a **provisional** rolling-7 baseline under a controlled trial, never as a production-validated forecasting service. Its evidence is one retrospective window.
- It carries no weather guard and no date-domain guard, because it takes neither input. Its one precondition is data completeness: seven consecutive prior daily counts must exist, and a gap in the series must produce an error rather than a mean over whatever rows happen to remain.
- It may **not** be described as machine learning, because it is not. Presenting a seven-day mean as a model would misrepresent the one honest finding this project has about forecasting.

C.6 What an operational forecasting service would require

Everything above describes the current artefact. A service a planner could actually rely on is a different piece of work, and the gap is worth stating so that it is not mistaken for a small one: recent multi-year demand rather than two years; archived weather **forecasts** rather than observed weather, so that forecast error is inside the measurement; rolling-origin validation across several forward windows; a declared forecast horizon; scheduled retraining; calibrated prediction intervals; monitoring against realised demand; and, as the gate, beating the rolling seven-day baseline forward in time.

Its supported window would then move with the model rather than being frozen at the assessment period:

```
{ "model_version": "2026-08-01",
  "trained_through": "2026-07-31",
  "forecast_horizon_days": 1,
  "supported_forecast_until": "2026-08-01" }
```

C.7 Geolocation and alerts

Two ideas worth recording, with their preconditions.

Station-level geolocation. Plotting expected demand onto a map through a mapping API is straightforward as an interface, and unsupported as an inference. This dataset carries no station identifier, coordinate, dock capacity or trip origin, so nothing in this notebook can attribute system demand to a location. Delivering it honestly means acquiring station-level data first; ITDP and NACTO describe the network, spacing and capacity variables that work would need (ITDP, 2018; NACTO, 2016).

Custom alerts. Two alert types are supported by evidence already collected.

A *domain alert* belongs to `/historical-estimate` alone, and fires when a request falls outside its supported weather categories or date range: the `unsupported_model_domain` response of C.3 recorded rather than merely returned. `/forecast` has no equivalent, because it accepts neither weather nor an arbitrary date. Its counterpart is a *completeness alert*, firing when the seven prior daily counts it needs are not all present.

A *drift alert* fires when rolling error exceeds a declared reference, and each method needs its own reference rather than a shared one:

Monitored	Reference	Value
`/forecast`, rolling-7	Its own 2012 temporal benchmark	MAE 847.8
Shadow model	Daily and rolling-window comparison against the served champion	relative, no fixed threshold
`/historical-estimate`	The random-holdout MAE it was evaluated against	MAE 433.9

A single crossing is noise rather than drift, so the alert needs a window and a tolerance as well as a reference. A workable starting rule for `/forecast` is a rolling 30-day MAE above 1.20 times 847.8 across two consecutive windows. Both constants are provisional: they are set here to make the mechanism concrete, and choosing what degradation is worth waking someone for is a stakeholder's decision, not a modelling one, exactly as with the demand threshold below.

The 433.9 figure must not become a forecasting threshold. It is the mean error of a conditional estimator over a random holdout drawn from both years, and applying it to day-ahead forecasting would be repeating the confusion between the two protocols that Section 4.2 exists to expose: a forecasting service held to it would alarm permanently, since no method in this study reached it going forward. This is the mechanism that would have caught the temporal failure in production rather than in a notebook, but only if it is pointed at the right number.

A demand-threshold alert for staffing would be useful but needs a stakeholder-defined threshold that this project does not have.

Academic Integrity Declaration

I declare that this submission is my own work. All sources of information, ideas and code have been acknowledged. The dataset is publicly available from the UCI Machine Learning Repository. The analysis, modelling decisions, interpretation and written commentary are my own.

Statement of Acknowledgement

Analysis was performed in Python with pandas, NumPy, scikit-learn, Matplotlib, Seaborn and SHAP.

I acknowledge that I have used the following AI tool(s) in the creation of this report:

- Anthropic Claude Opus 5
- OpenAI ChatGPT Codex 5.6

Both tools were used to assist with understanding ML concepts, challenging the experimental design, auditing empirical claims against executed outputs, structuring the technical pipeline, improving clarity of academic language, and supporting APA 7th referencing conventions.

Prompt examples:

1. "Explain why a random split over a two-year time series supports a conditional demand estimate but not a next-day forecast, and how to run both experiments so that neither contaminates the other."
2. "Is a gradient boosting ensemble mathematically bounded by its training target range the way a random forest is? Give me a counterexample if it is not, and tell me which of my claims it breaks."
3. "Build a causal seven-day rolling baseline for a day-ahead comparison using shift before rolling, and write the assertion that proves the window never includes the target day."

I confirm that the use of these tools has been in accordance with the Torrens University Australia Academic Integrity Policy and TUA, Think and MDS's Position Paper on the Use of AI. I confirm that the final output is authored by me and represents my own critical thinking, analysis, and synthesis of sources. I take full responsibility for the final content of this report.

References

Capital Bikeshare. (n.d.). *How it works*. https://capitalbikeshare.com/how-it-works

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.

Fanaee-T, H., & Gama, J. (2014). Event labeling combining ensemble detectors and background knowledge. *Progress in Artificial Intelligence, 2*(2-3), 113-127. https://doi.org/10.1007/s13748-013-0040-3

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189-1232. https://doi.org/10.1214/aos/1013203451

Institute for Transportation & Development Policy. (2018). *The bikeshare planning guide*. https://itdp.org/publication/the-bike-share-planning-guide/

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems, 30*, 4765-4774.

National Association of City Transportation Officials. (2016). *Bike share station siting guide*. https://nacto.org/publication/bike-share-station-siting-guide/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

University of California, Irvine. (n.d.). *Bike Sharing Dataset*. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset